레볼루션 1일차

레볼루션 동아리 오픈 클래스

21학번 민대영

목표:

3D 월드에서
직접 공을 움직여보기
총 쏴보기

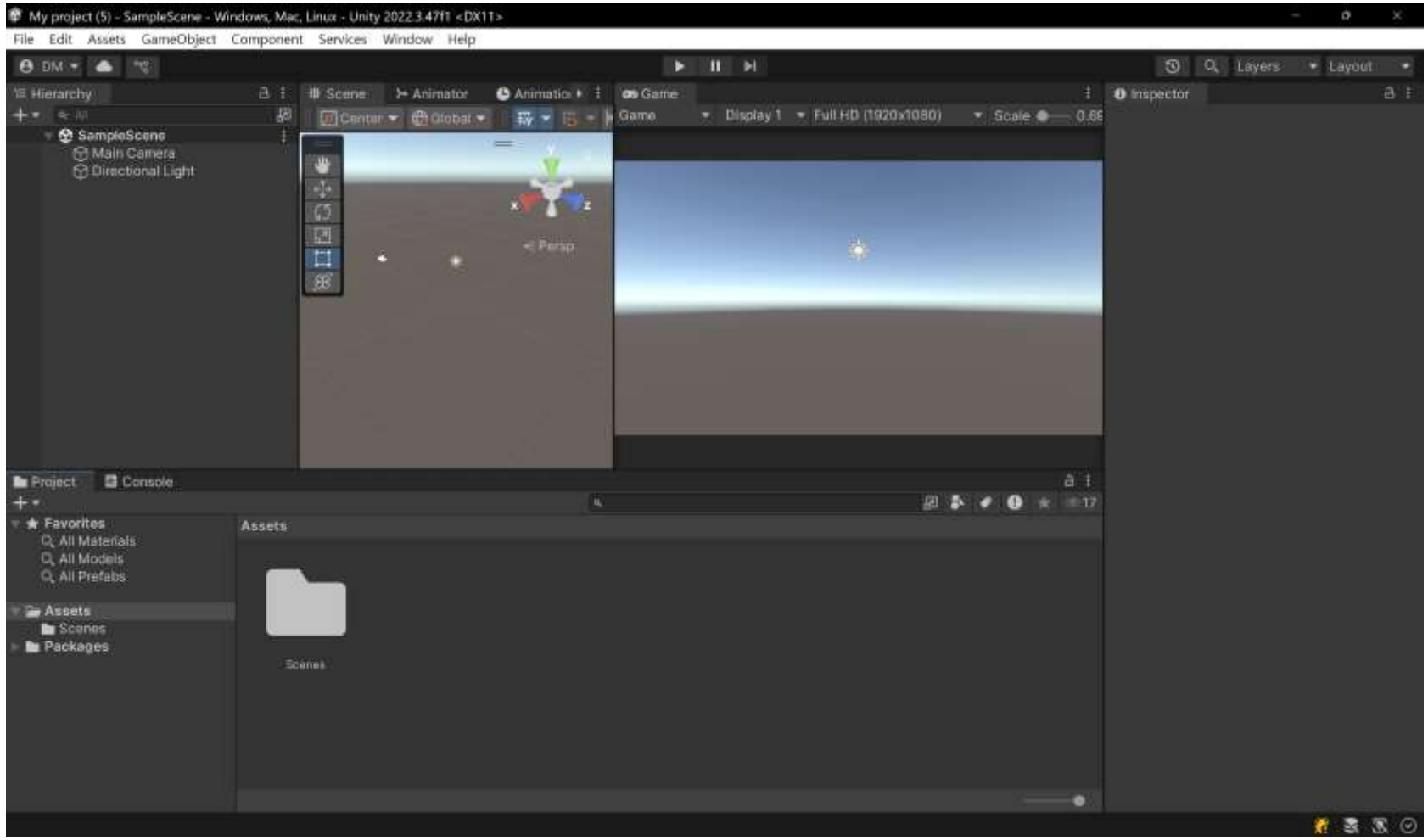
학습:

씬 뷰 조작법
스크립트
컴포넌트
프리팹
Ray
인스턴시에이션

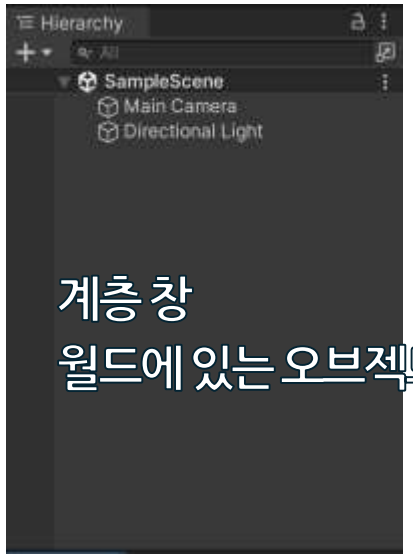
상세설명 링크 (Ctrl+F로 번호 찾기 ex.1-1)

https://jumpy-lifeboat-4f2.notion.site/1ed36fa4b4478085af8bd8a9edfb068c?pvs=4

유니티 기본 화면

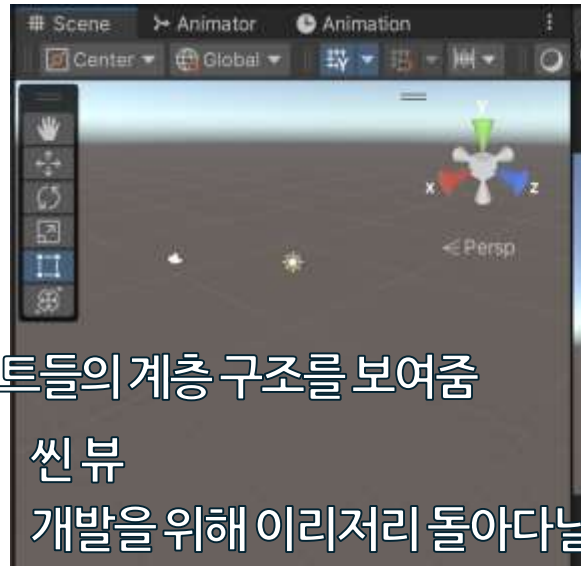

My project (5) - SampleScene - Windows, Mac, Linux - Unity 2022.3.47f1 <DX11>
File
Edit
Assets
GameObject
Component
Services
Window
Help
DM
Layers
Layout
Hierarchy
SampleScene
Main Camera
Directional Light
Scene
Animator
Animatio
Center
Global
Persp
Game
Display 1
Full HD (1920x1080)
Scale
Inspector
Project
Console
Favorites
All Materials
All Models
All Prefabs
Assets
Scenes
Packages
Assets
Scenes

소개



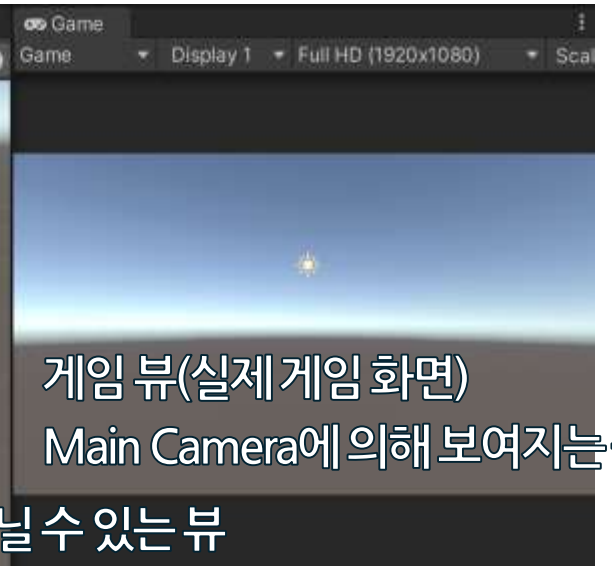
계층 창

월드에 있는 오브젝트들의 계층 구조를 보여줌



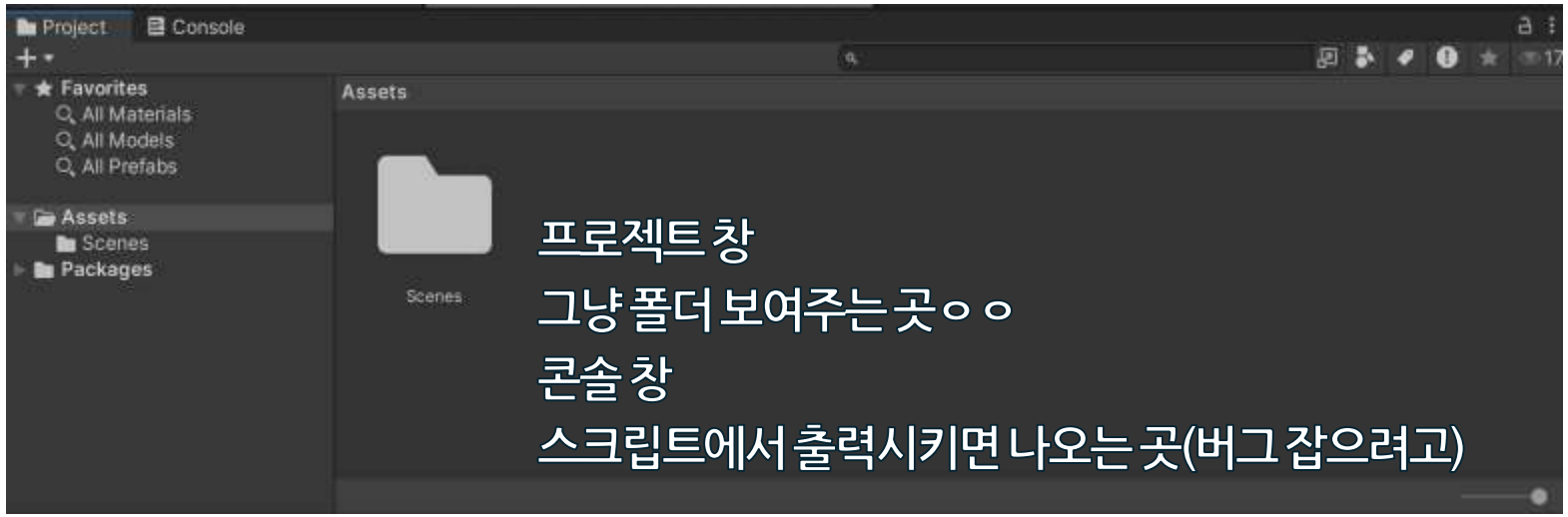
씬 뷰

개발을 위해 이리저리 돌아다닐 수 있는 뷰



게임 뷰(실제 게임 화면)

Main Camera에 의해 보여지는 곳

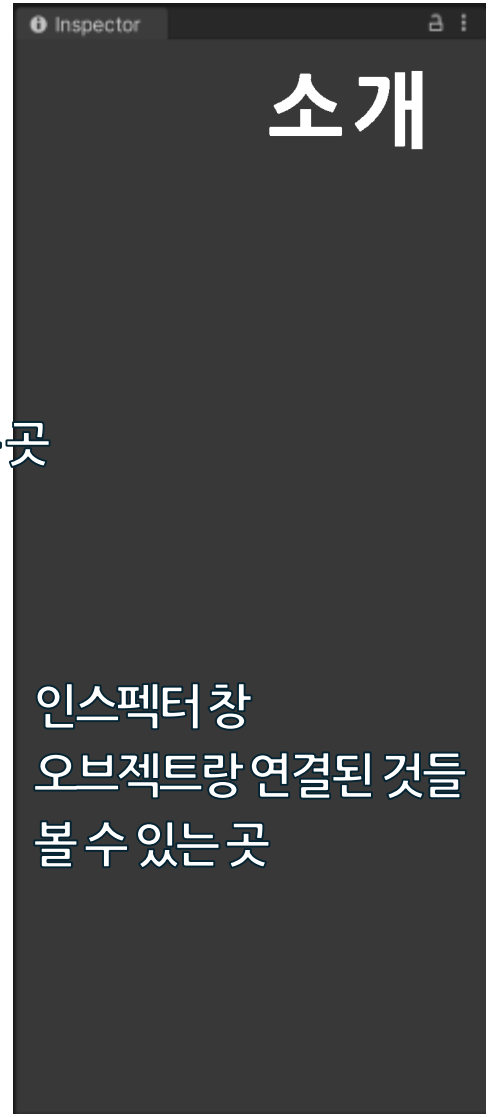


프로젝트 창

그냥 폴더 보여주는 곳ㅇㅇ

콘솔 창

스크립트에서 출력시키면 나오는 곳(버그 잡으려고)

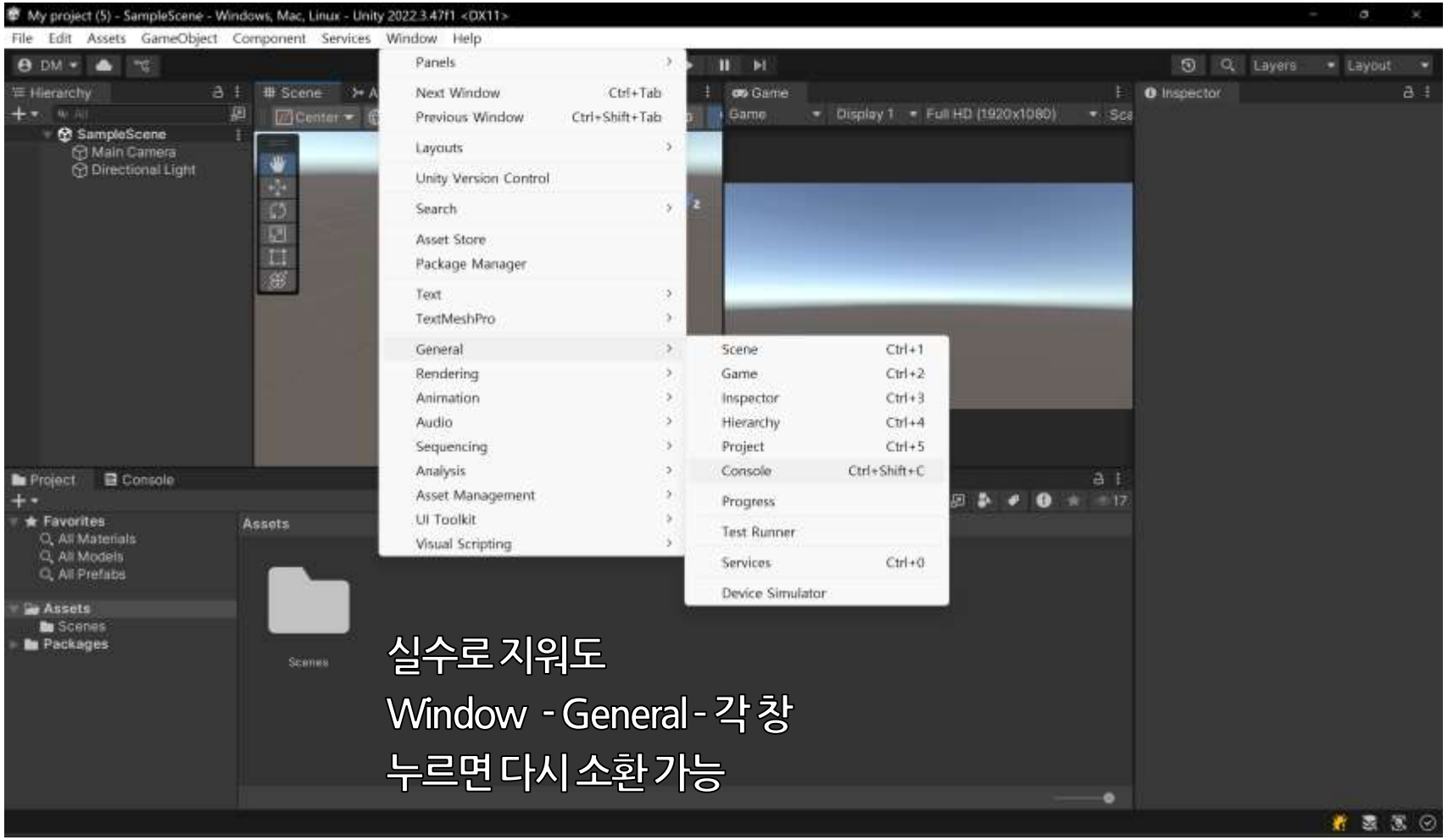


인스펙터 창

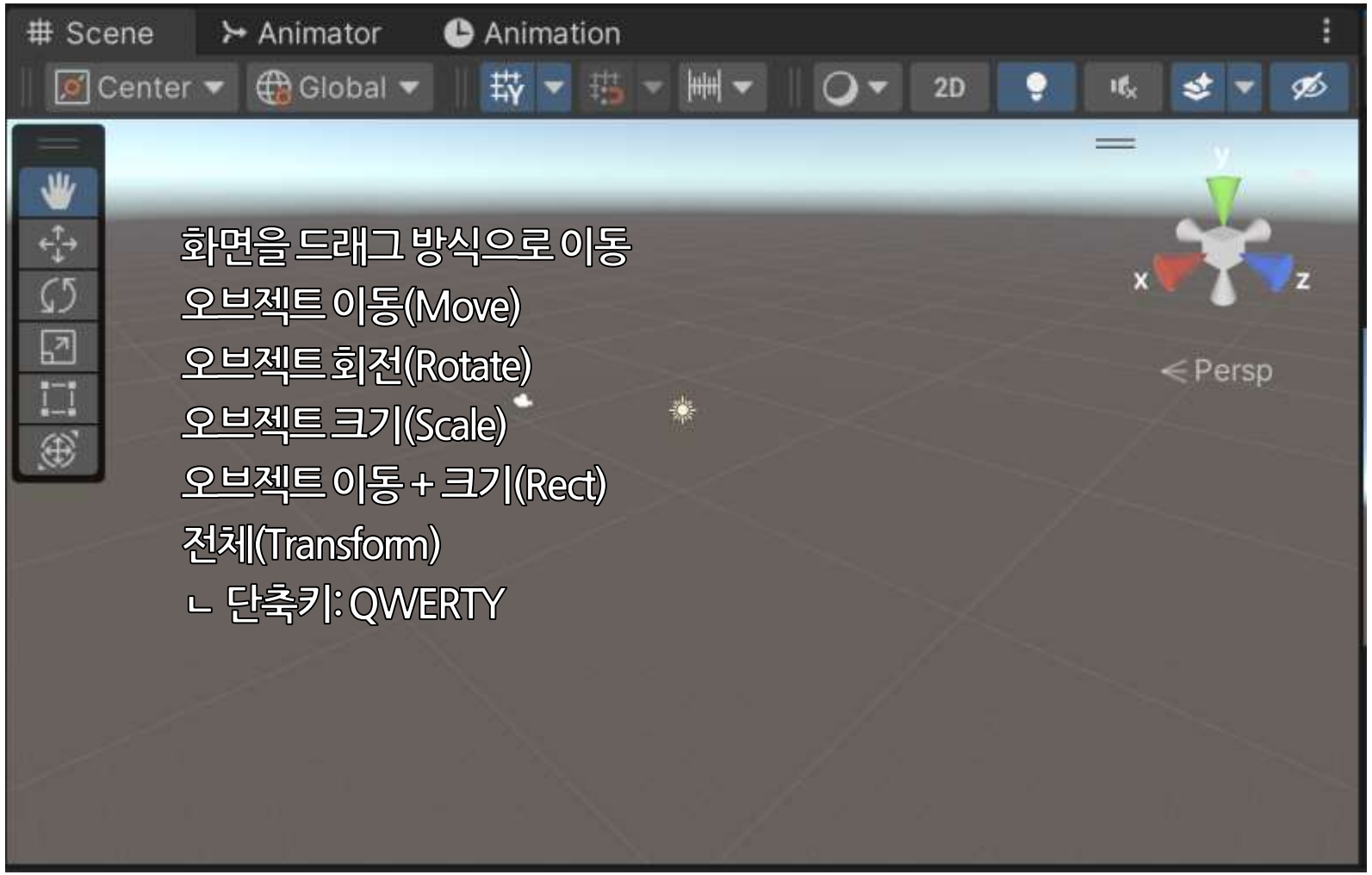
오브젝트랑 연결된 것들

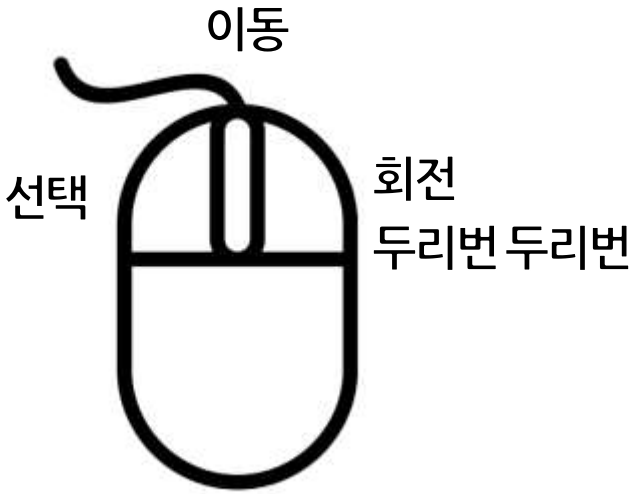
볼 수 있는 곳

다시 소환하는 법

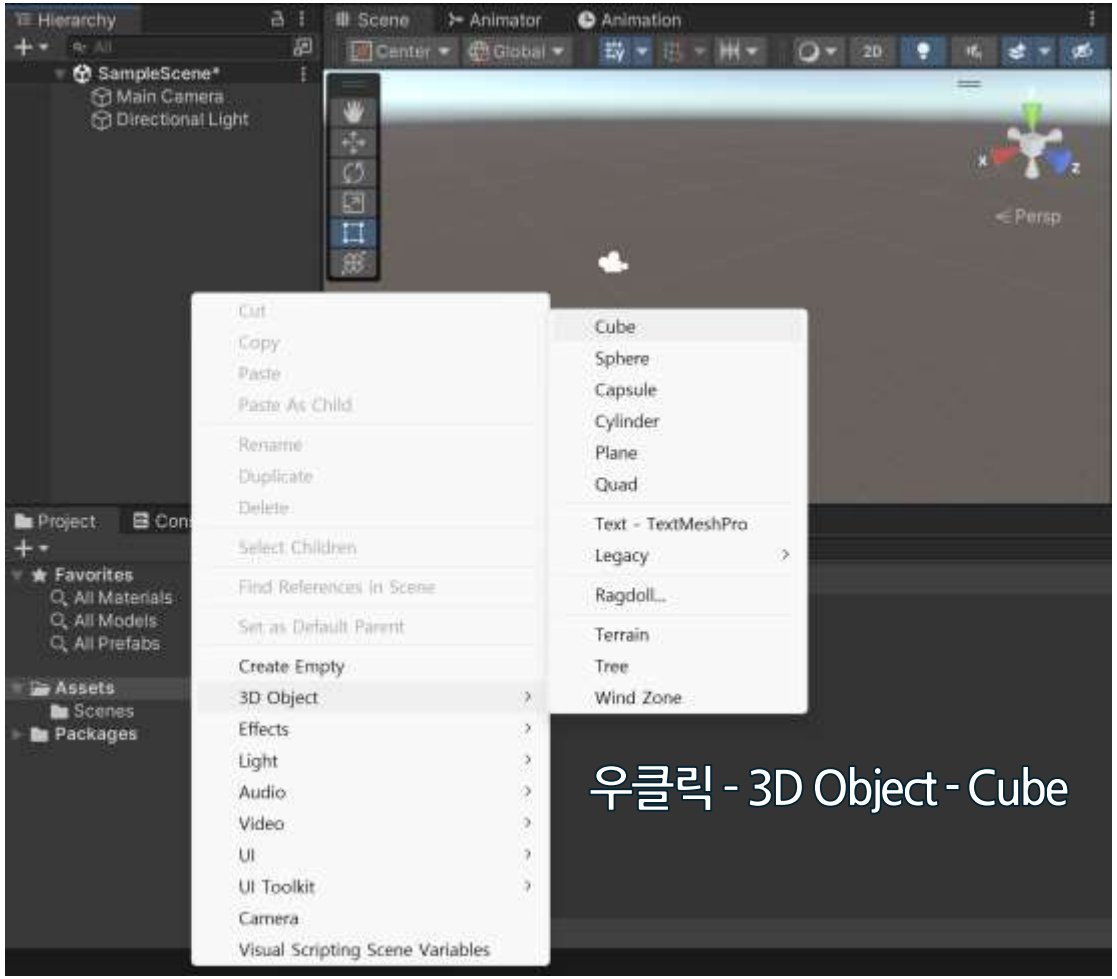

My project (5) - SampleScene - Windows, Mac, Linux - Unity 2022.3.47f1 <DX11>
File Edit Assets GameObject Component Services Window Help
Panels
Next Window Ctrl+Tab
Previous Window Ctrl+Shift+Tab
Layouts
Unity Version Control
Search
Asset Store
Package Manager
Text
TextMeshPro
General
Rendering
Animation
Audio
Sequencing
Analysis
Asset Management
UI Toolkit
Visual Scripting
Scene Ctrl+1
Game Ctrl+2
Inspector Ctrl+3
Hierarchy Ctrl+4
Project Ctrl+5
Console Ctrl+Shift+C
Progress
Test Runner
Services Ctrl+0
Device Simulator
실수로 지워도
Window - General - 각 창
누르면 다시 소환 가능

씬 뷰 조작 방법

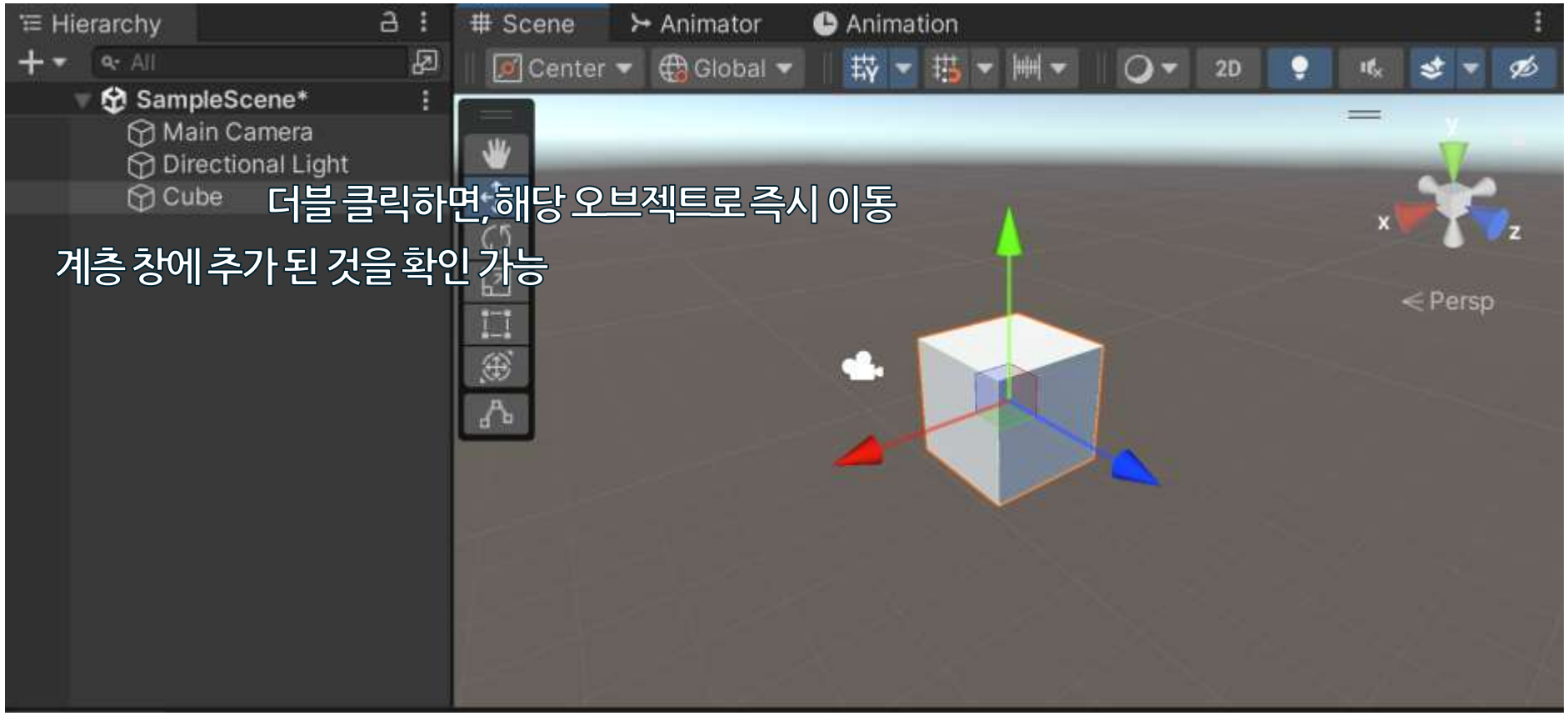

Scene
Animator
Animation
Center
Global
2D
y
x
z
Persp
화면을 드래그 방식으로 이동
오브젝트 이동(Move)
오브젝트 회전(Rotate)
오브젝트 크기(Scale)
오브젝트 이동 + 크기(Rect)
전체(Transform)
ㄴ 단축키: QWERTY


이동
선택
회전
두리번 두리번

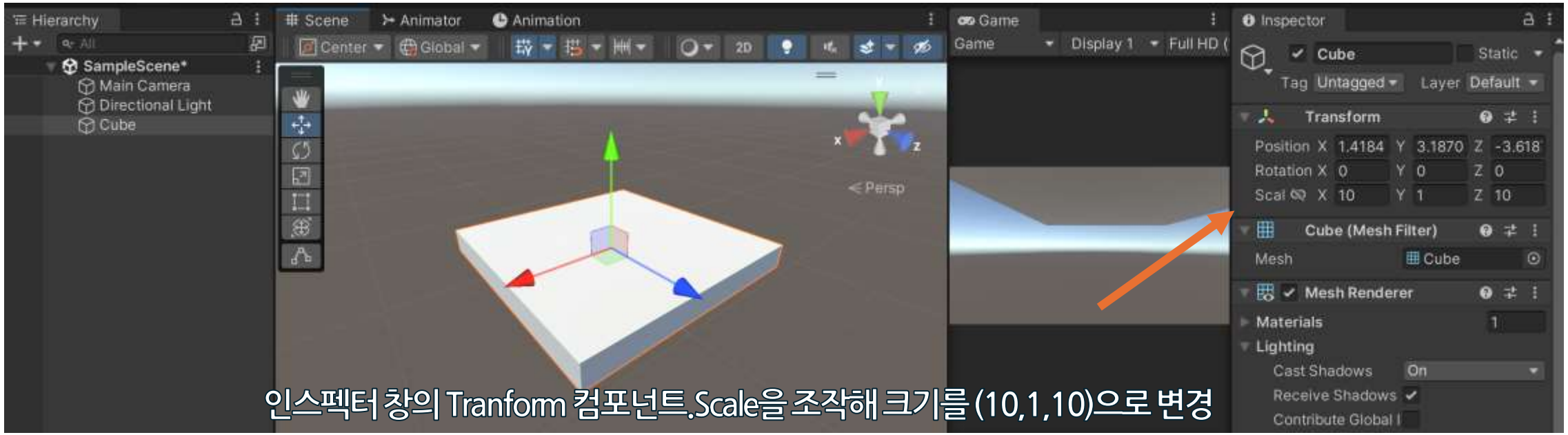
오브젝트를 추가해보자


Hierarchy
Scene
Animator
Animation
Center
Global
2D
SampleScene*
Main Camera
Directional Light
Persp
Cut
Copy
Paste
Paste As Child
Rename
Duplicate
Delete
Select Children
Find References In Scene
Set as Default Parent
Create Empty
3D Object
Effects
Light
Audio
Video
UI
UI Toolkit
Camera
Visual Scripting Scene Variables
Cube
Sphere
Capsule
Cylinder
Plane
Quad
Text - TextMeshPro
Legacy
Ragdoll...
Terrain
Tree
Wind Zone
Project
Favorites
All Materials
All Models
All Prefabs
Assets
Scenes
Packages
우클릭 - 3D Object - Cube

앞의 조작법을 참고해 이리저리 움직여보자



큐브를 땅으로 만들기



인스펙터 창의 Tranform 컴포넌트.Scale을 조작해 크기를 (10,1,10)으로 변경

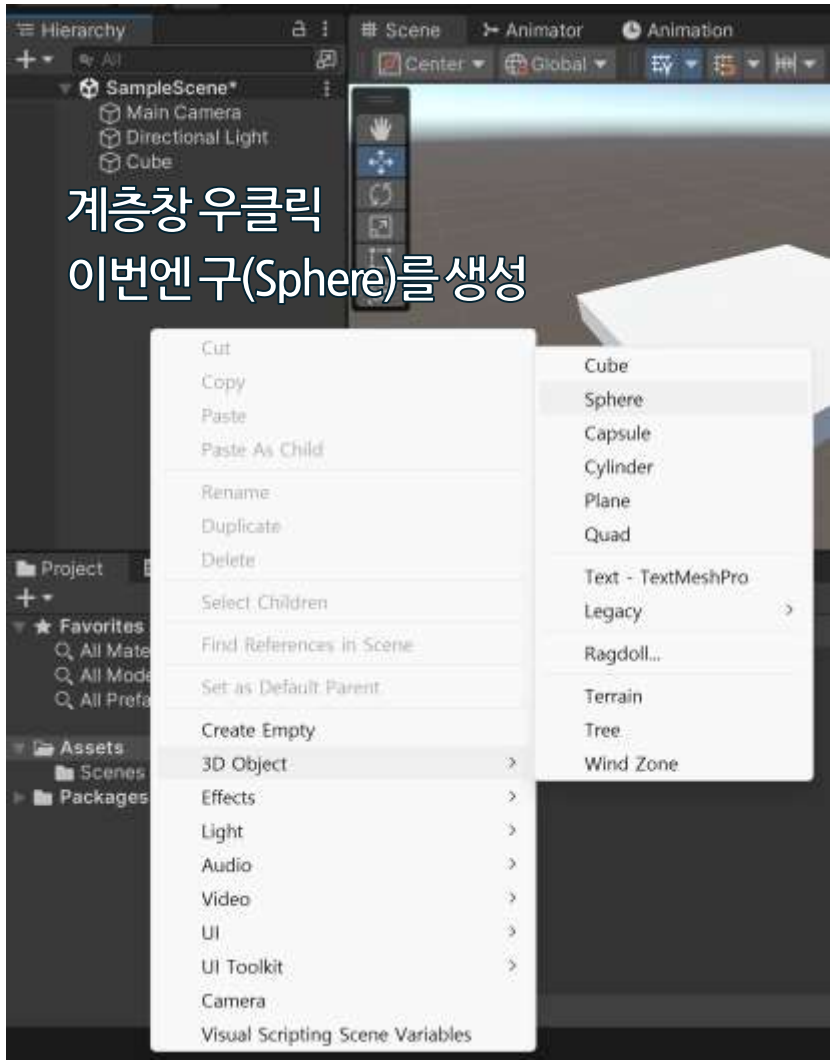
*컴포넌트(Component): 게임 오브젝트에 부착되어 동작을 담당하는 모듈

Transform컴포넌트는 게임 오브젝트의 위치, 회전, 크기를 저장하는 모듈인 것

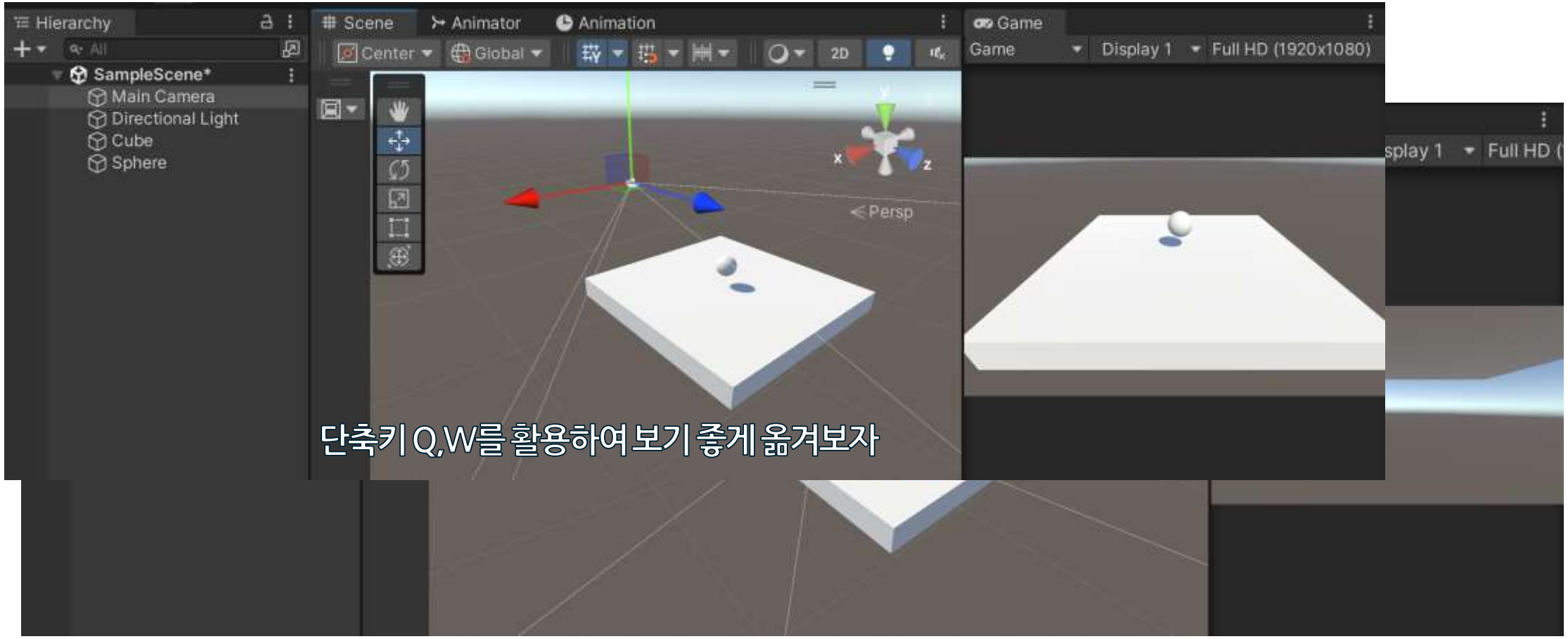
이는 나중에 스크립트에서도 조작이 가능하다!

인스펙터 창은 코드(스크립트)의 내용을 보기 쉽게 출력만 하는 형태임

구 생성하기


Hierarchy
Scene
Animator
Animation
Center
Global
SampleScene*
Main Camera
Directional Light
Cube
계층창 우클릭
이번엔 구(Sphere)를 생성
Cut
Copy
Paste
Paste As Child
Rename
Duplicate
Delete
Select Children
Find References in Scene
Set as Default Parent
Create Empty
3D Object
Effects
Light
Audio
Video
UI
UI Toolkit
Camera
Visual Scripting Scene Variables
Cube
Sphere
Capsule
Cylinder
Plane
Quad
Text - TextMeshPro
Legacy
Ragdoll...
Terrain
Tree
Wind Zone
Project
Favorites
Assets
Scenes
Packages

잘 안 보이니까 카메라 위치 이동시키기

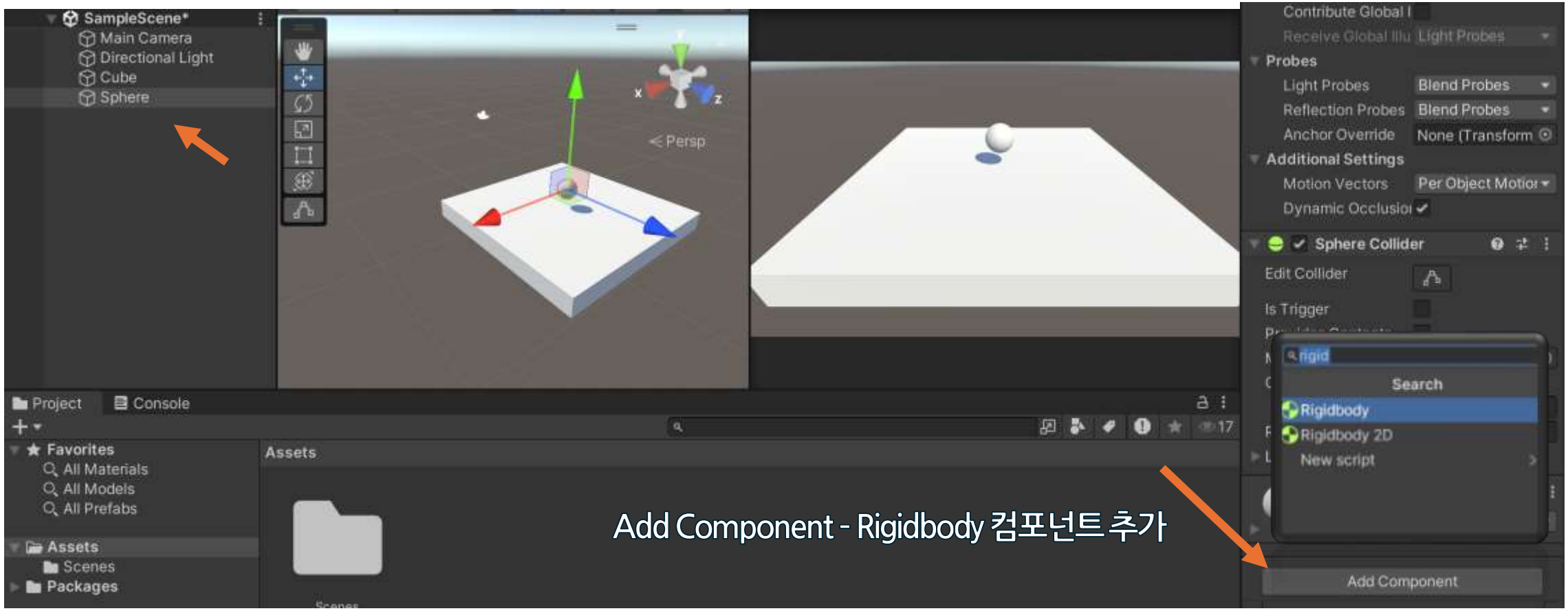


게임 뷰 실행해보기

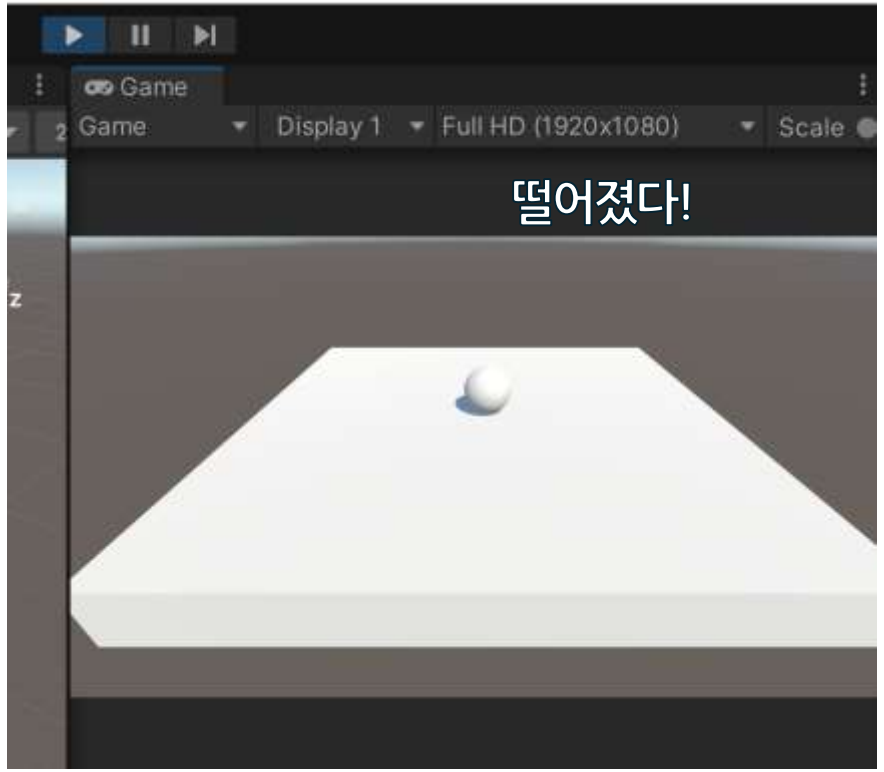
실행/일시정지/버그 잡기용 Step(한 프레임씩만 실행)

실행해보았는데, 그냥 떠있기만 함..
우리가 원하는 게 아님

중력 적용되는 컴포넌트 추가하기



실행하여 잘 적용된 것을 확인 가능



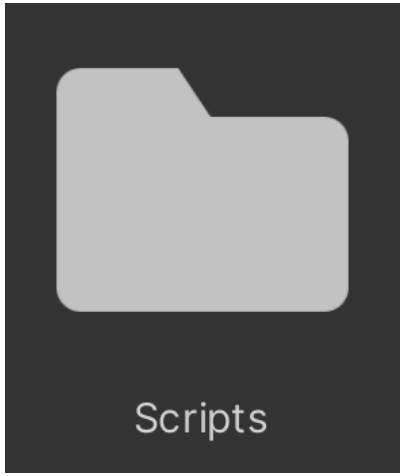
이제 움직이기 위해 스크립트를 적용해보자

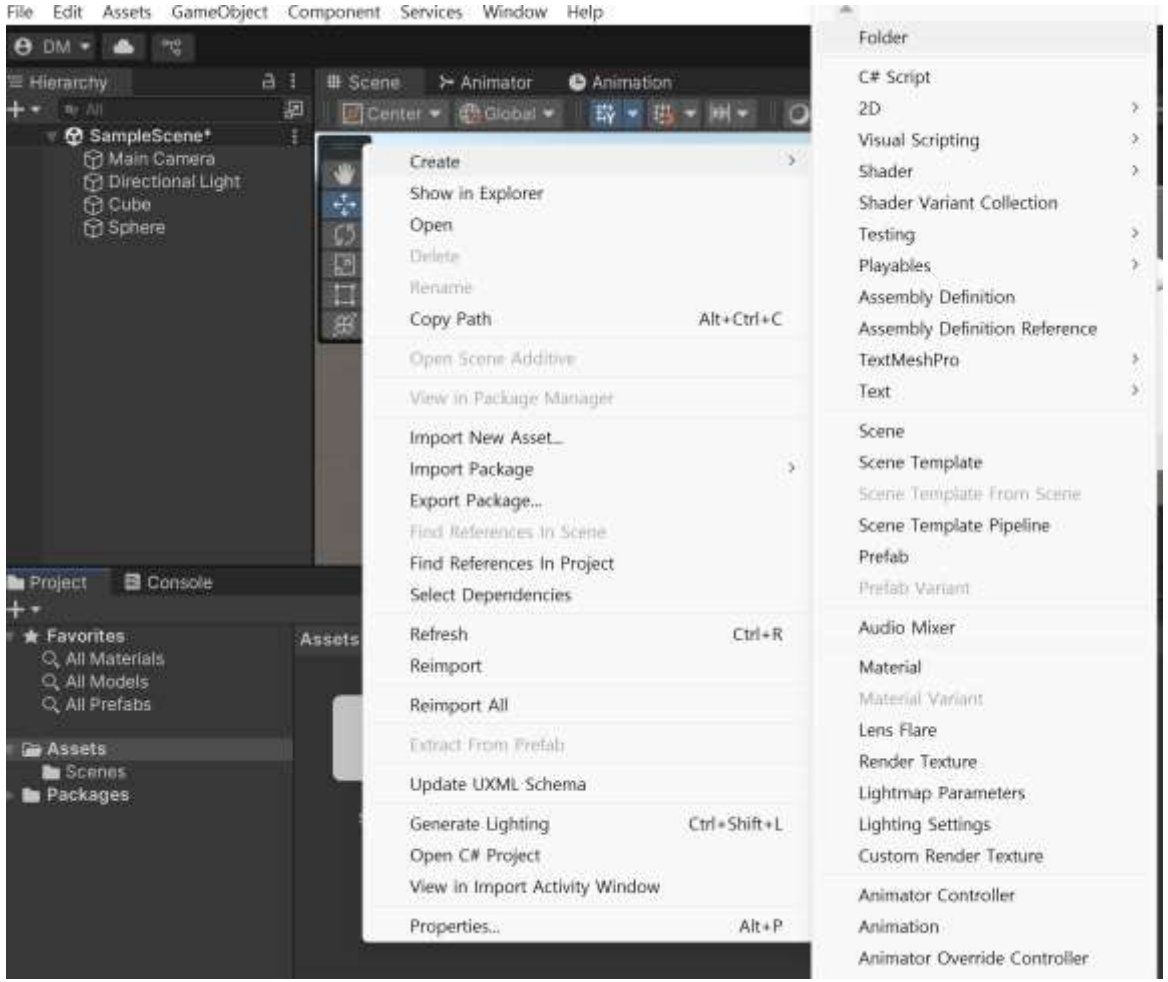
먼저 스크립트를 넣을 폴더부터 생성

프로젝트 창 우클릭- Create - Folder

Folder 이름: **Scripts**

너무 의미X 걍 스크립트 넣는 폴더니까 Scripts

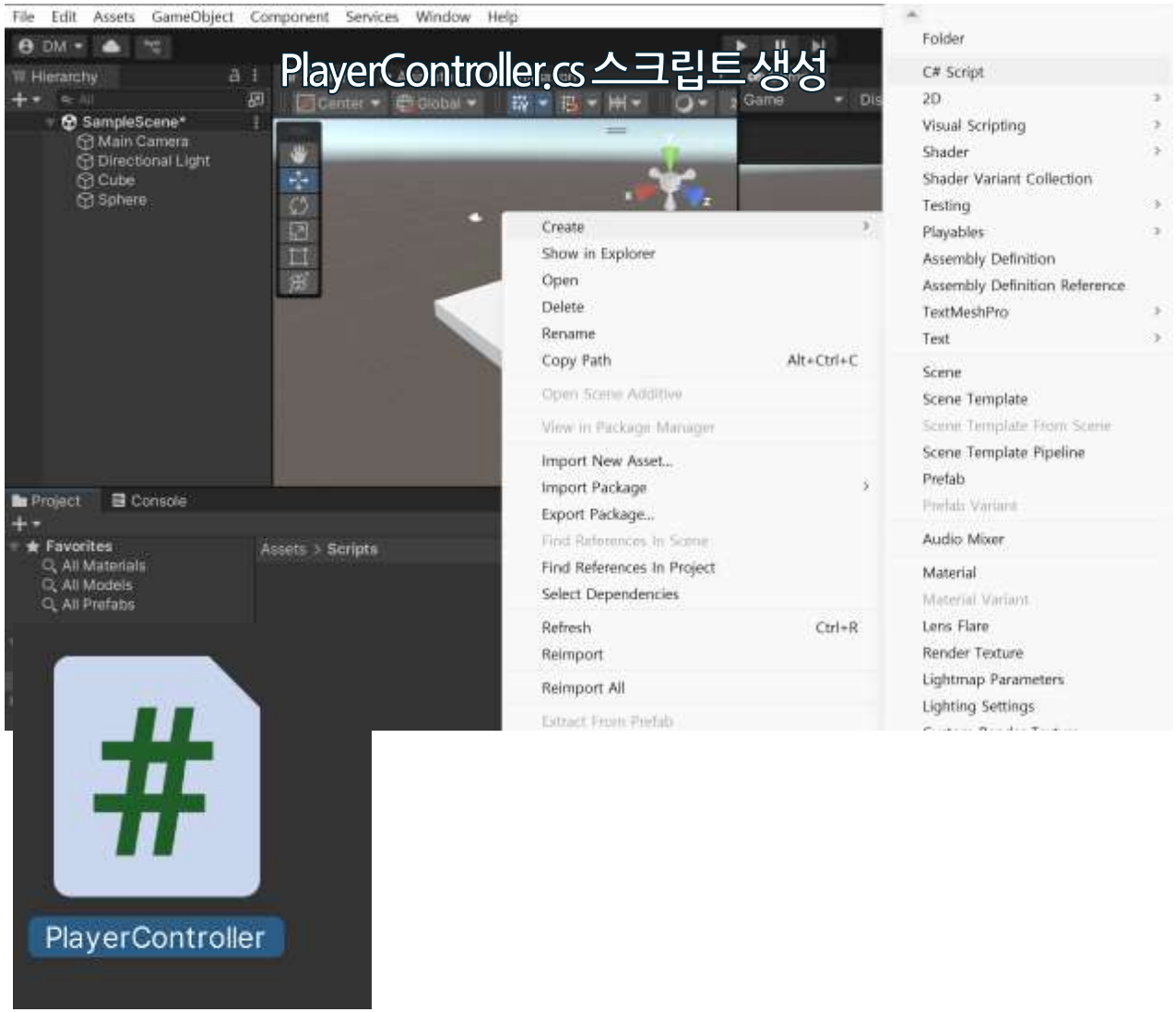




스크립트 생성

폴더 내에 C# 스크립트를 생성하자
프로젝트 창 - 우클릭
- Create - C# Script
Script 이름: PlayerController.cs

스크립트 이름에 주의하자!
스크립트 이름과 클래스 이름은 항상 같아야 한다!
유니티가 해당 코드를 인식하기 위함!



스크립트 확인

PlayerController.cs × 확장: C#

스크립트 이름과 똑같다!

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class PlayerController : MonoBehaviour
{
    // Start is called before the first frame update
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {
```

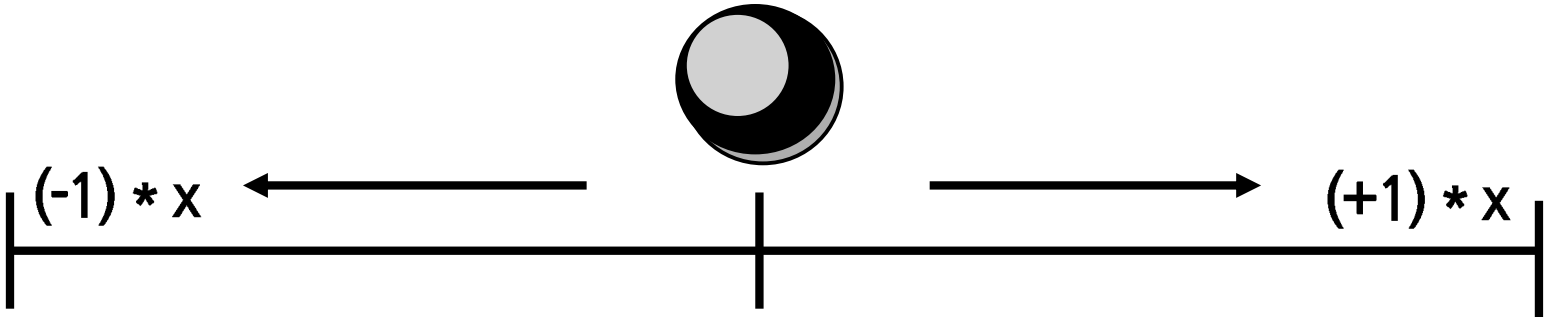
클래스 이름이 PlayerController인 것을 확인 가능

스크립트 구성

지금부터 만들 스크립트의 정체는 **플레이어 조종장치!**
플레이어가 **방향키**를 누르면, 해당 게임 오브젝트가 **왼쪽**이나 **오른쪽**으로 움직이게 만들어 줄 것임. ← →

주요 흐름

1. 플레이어의 **좌우 입력**을 **키보드**나 **조이스틱**으로 받음.
2. 입력에 따른 **이동거리**를 **계산**함.
3. 해당 게임 오브젝트를 이동거리만큼 **이동**시킴.



코드

```
public class PlayerController : MonoBehaviour
{
    float speed = 5f;

    // Update is called once per frame
    void Update()
    {
        float horizontalInput = Input.GetAxis("Horizontal");
        float moveDistance = horizontalInput * Time.deltaTime * speed;
        this.GetComponent<Transform>().position += new Vector3(moveDistance, 0, 0);
    }
}
```

좌우 이동 1-1

코드

```
void Update()
```

매 프레임마다 **실행**되는 함수다. 게임 오브젝트의 주요 동작은 다 여기서 이루어진다.
만약 게임이 60fps(frame per second)라면, 이 함수가 매초마다 60번씩 실행되는 것

```
float horizontalInput = Input.GetAxis("Horizontal");
```

왼쪽이나 오른쪽 키를 눌렀는지 확인하고, 그 **방향에 따라 (-1) ~ 1 사이의 숫자**를 알려줌
ex. **왼쪽**이면 **-1**, 아무 것도 안 누르면 0, **오른쪽**이면 **1**

```
float moveDistance = horizontalInput * Time.deltaTime * speed;
```

Time.deltaTime: 프레임 당 걸린 시간
게임이 항상 일정하게 동작할 수 있도록 deltaTime을 활용하여 이동거리를 측정함.
(∵속력 = 이동거리/시간, **이동거리 = 속력 *시간**, 추가로 이동 방향까지 곱해줌)

코드

```
this.GetComponent<Transform>().position += new Vector3(moveDistance, 0, 0);
```

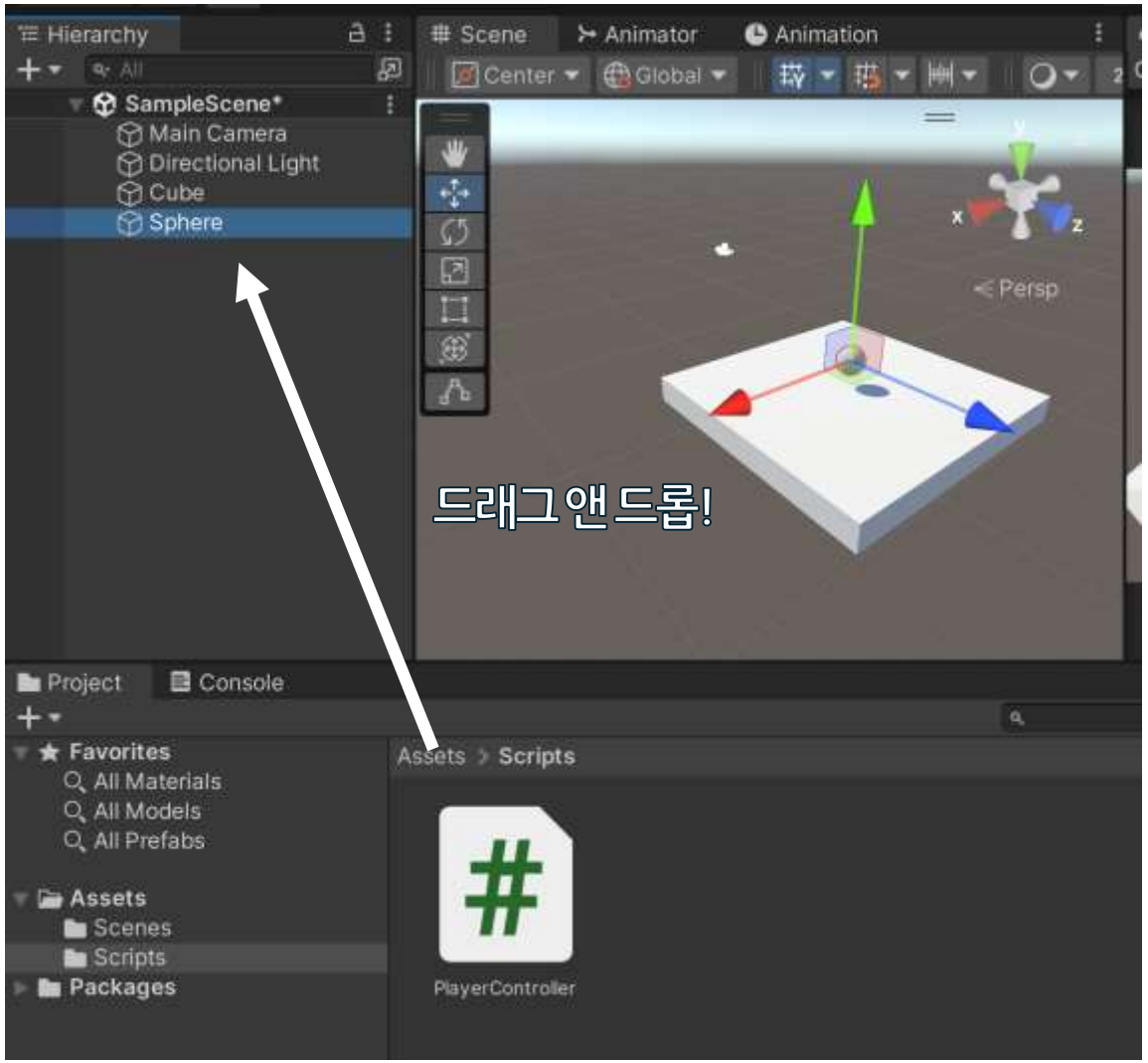
```
this.GetComponent<Transform>()
```

이 **스크립트**가 붙은 **게임 오브젝트**가 사용 중인 바로 그 **Transform 컴포넌트**를 반환함.

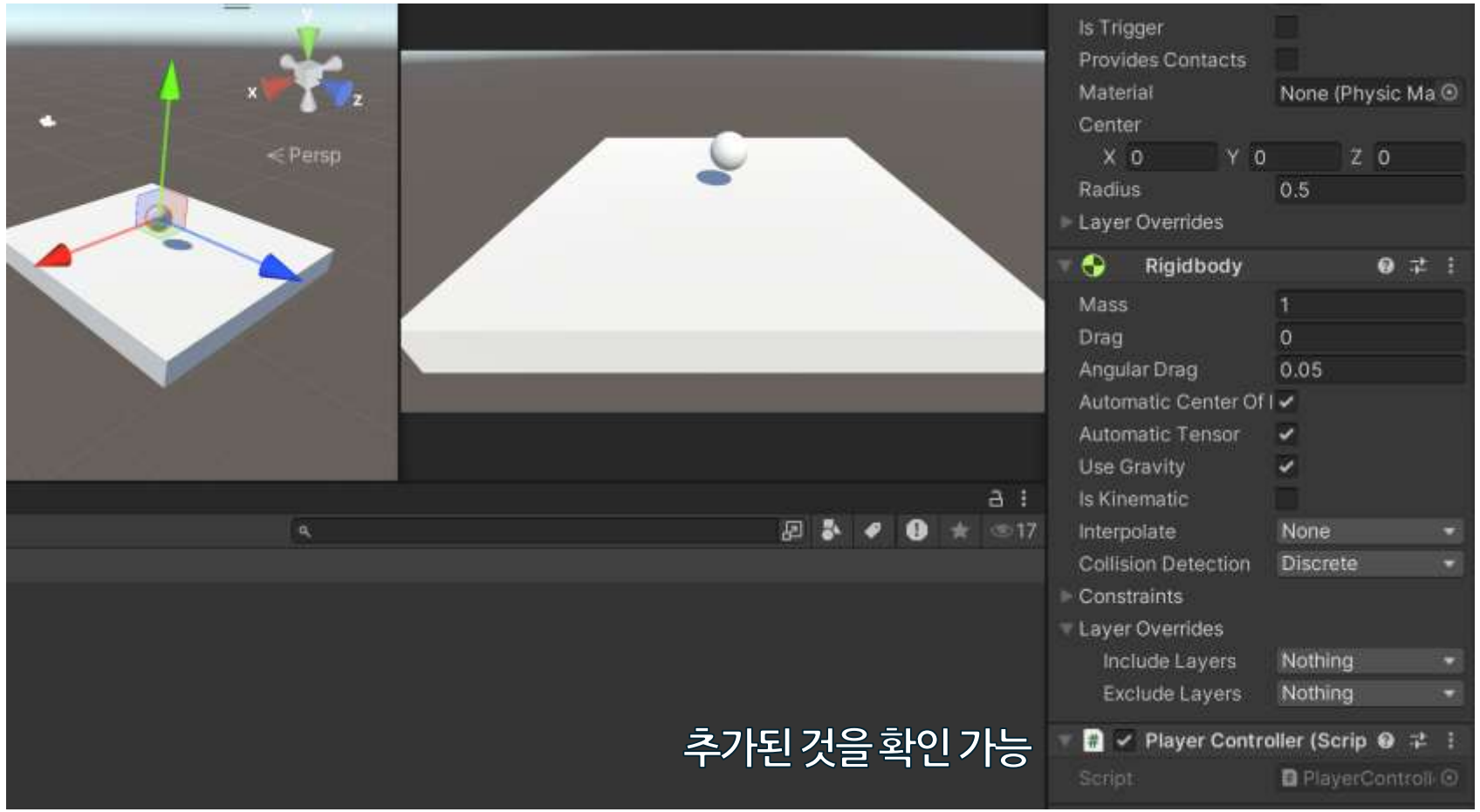
```
.position += new Vector3(moveDistance, 0, 0);
```

그 컴포넌트가 관리 중인 position 변수의 값을 내가 이동거리만큼 바꿔버리겠어
Position의 타입은 Vector3니까 Vector3를 만들어서 전달해줘야함

게임 오브젝트에 붙이자!

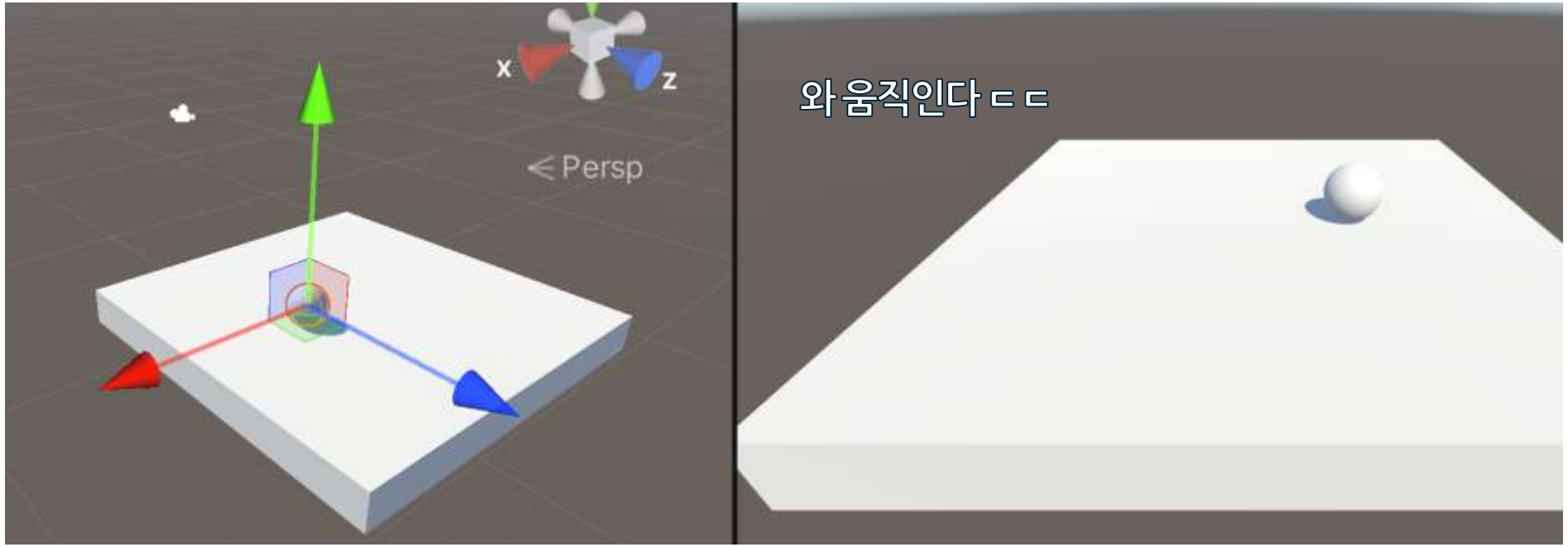

Hierarchy
Scene
Animator
Animation
Center
Global
SampleScene*
Main Camera
Directional Light
Cube
Sphere
Persp
드래그 앤 드롭!
Project
Console
Favorites
All Materials
All Models
All Prefabs
Assets
Scenes
Scripts
Packages
Assets > Scripts
PlayerController

게임 오브젝트에 붙였다!



추가된 것을 확인 가능

실행해보면, 움직일 수 있음

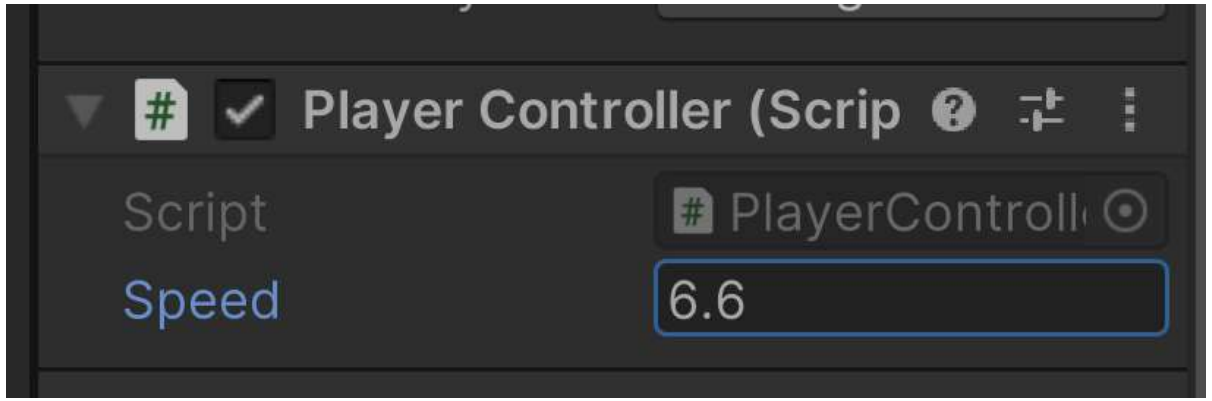


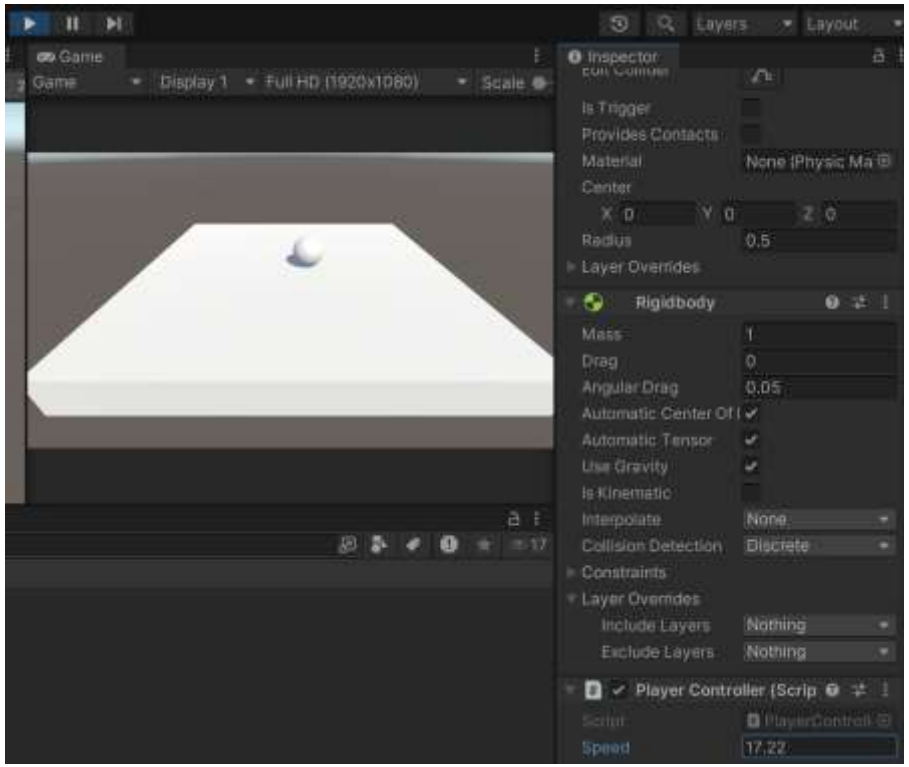
속도 변수 앞에 public을 추가해보자

```
public class PlayerController : MonoBehaviour
{
    public float speed = 5f;

    // Update is called once per frame
    void Update()
```

public : 어디서든 접근 가능
private : 나만 접근 가능

이제 인스펙터 창에서 해당 변수를 바꿀 수 있다





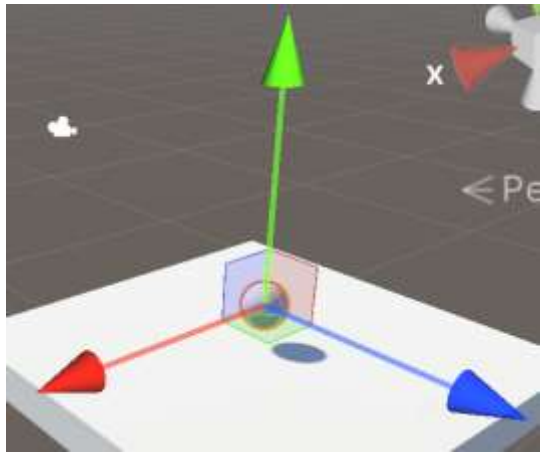
심지어 실행 도중에도 변경 가능

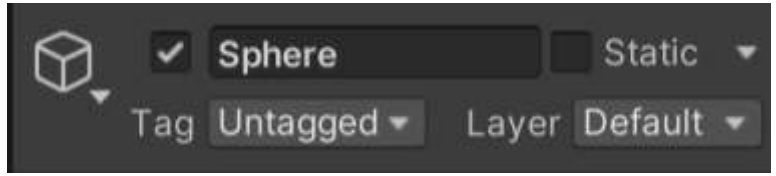
대신 버그 잡을 때 힘들어지겠죠? 어디서든 바꿀 수 있으니까?

유니티의 처음이자 끝! 컴포넌트

```
this.GetComponent<Transform>().position += new Vector3(moveDistance, 0, 0);
```

```
this.GetComponent<Transform>()
```





ex)
SpriteRenderer: 2D 이미지 컴포넌트
TMP_Text: 텍스트 컴포넌트
Collider: 충돌 판정 컴포넌트

그렇다! 사실 **컴포넌트**의 정체는 잘 정돈된 **스크립트**

스크립트 == 컴포넌트이므로 `this.GetComponent<PlayerController>()` 처럼 직접 만든 스크립트를 컴포넌트로 받아오는 게 가능하다.

나중에 스크립트끼리 소통하는 법도 배워보자

최적화를 하자면, 이와 같다

```
// 기존 코드
this.GetComponent<Transform>().position = new Vector3(moveDistance, 0, 0);
// ↓ this는 생략 가능
GetComponent<Transform>().position = new Vector3(moveDistance, 0, 0);
// ↓ transform은 널리 쓰이는 매우 기초적인 컴포넌트이기 때문에 기본으로 제공이 됨.
transform.position += new Vector3(moveDistance, 0, 0);
/*
자주 쓰이는 기본 제공 키워드는 아래와 같음
    trasform
    gameObject: 현재 이 스크립트가 붙은 게임 오브젝트
    tag: 태그, 이후 배울 거
    name: 오브젝트 이름
*/
```

위 3코드는 똑같은 녀석을 가리키고 있음.

이제 상하좌우 이동하면서 점프까지 하게 만들자

```
public class PlayerController : MonoBehaviour
{
    public float speed = 5f;

    // Update is called once per frame
    void Update()
    {
        float horizontalInput = Input.GetAxis("Horizontal");
        float moveDistance = horizontalInput * Time.deltaTime * speed;
        transform.position += new Vector3(moveDistance, 0, 0);
    }
}
```

이건 기존 좌우이동 코드.

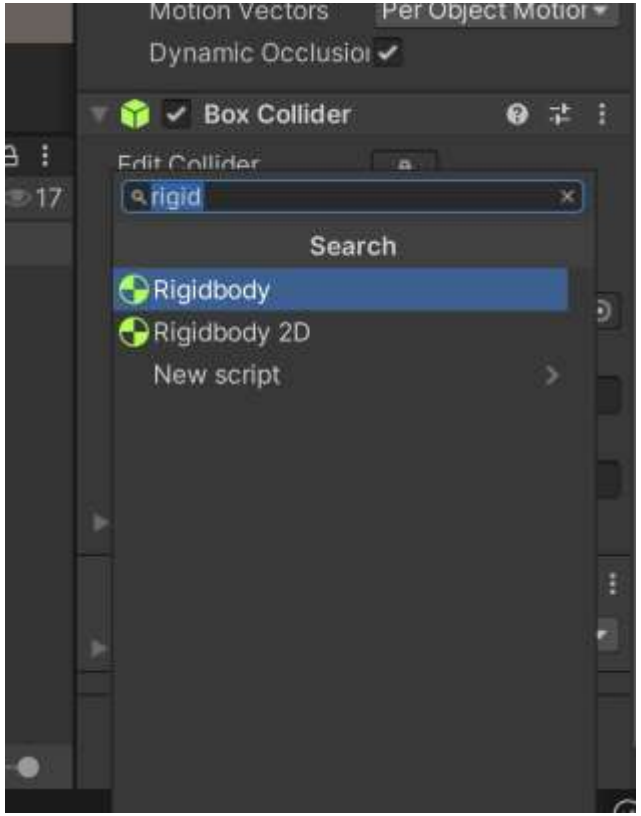
상하 이동과 **점프 기능**을 만들어야 한다.

점프 기능도 단순 위치 변환으로 직접 구현할 수 있지만, 시간이 소요되니 **유니티**의 **물리 엔진**을 사용해보자

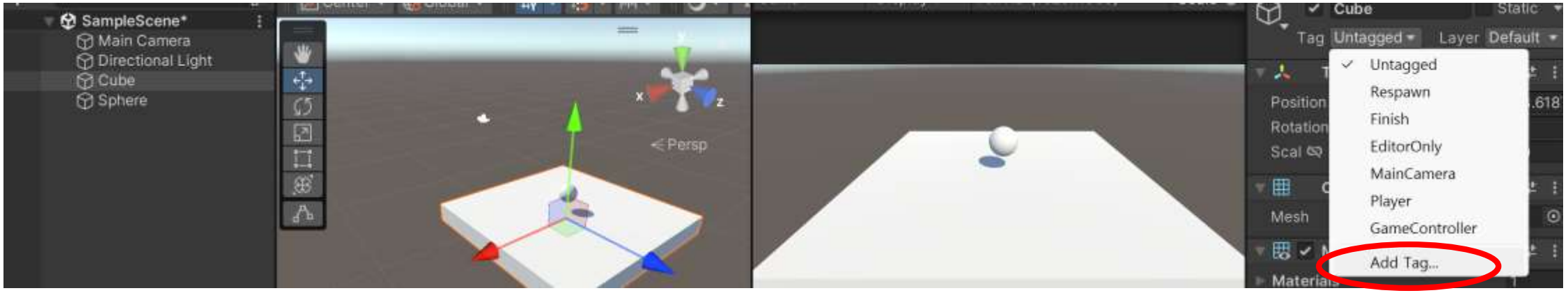
일단 물리적 영향을 받도록 컴포넌트를 추가하자

Rigidbody는 게임 오브젝트에 물리 엔진을 적용하여 실제 물체의 움직임처럼 힘과 토크의 영향을 받는 컴포넌트입니다. 즉, 중력, 질량, 속도, 가속도 등의 물리적 특성을 부여하고, 외부 힘이나 충돌에 의해 오브젝트가 사실적으로 움직이도록 합니다. *-Unity매뉴얼*

Add Component 버튼을 눌러 Rigidbody컴포넌트를 추가하자

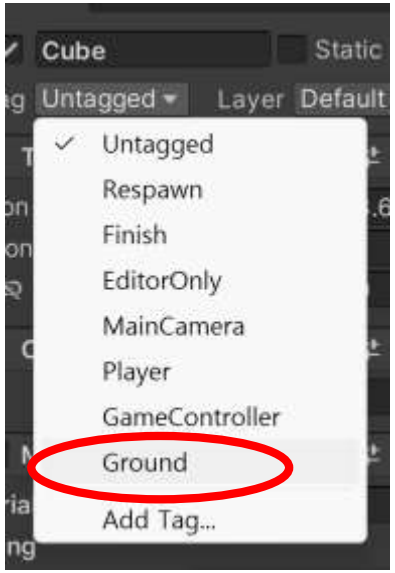


어디가 땅인지 구분하기 위해 tag를 설정하자



Add Tag 선택

+클릭 후 태그 이름 Ground 추가



Cube의 태그를 Ground로 설정

변경 코드 구성

매 프레임마다

1. 점프 입력을 받으면 점프한다.
2. 이동 입력을 받으면 이동한다.

점프

1. 지금 땅 위라면, 점프 한다.
2. 점프는 지면을 박차고 위로 이동하는 것을 뜻한다.

이동

1. 좌우 입력을 받는다.
2. 상하 입력을 받는다.
3. 상하좌우 입력 방향으로 위치를 변환한다,

충돌 판단

1. 현재 충돌 중인 게임 오브젝트의 태그가 '땅' 이라면, 현재 땅 위에 있음을 인정한다.

전체 코드

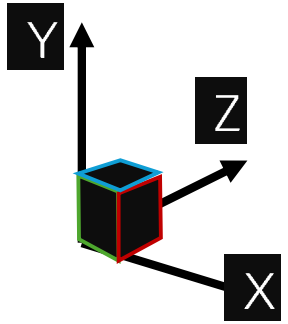
```
public class PlayerController : MonoBehaviour
{
    private Rigidbody rigid;

    float speed = 5f;
    float jumpPower = 8f;
    bool isGround;

    void Start()
    {
        isGround = false;
        rigid = GetComponent<Rigidbody>();
    }

    // Update is called once per frame
    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Space))
        {
            Jump();
        }
        Move();
    }
```

← 컴포넌트는 컴포넌트이기 전에 클래스, 즉 스크립트이므로 **변수로 저장 가능**

← 앞에서 이미 컴포넌트를 추가했기 때문에 시작할 때 받아올 수 있음

전체 코드

```
    void Move()
    {
        float horizontalInput = Input.GetAxis("Horizontal");
        float verticalInput = Input.GetAxis("Vertical");
        Vector3 direction = new Vector3(horizontalInput, 0, verticalInput).normalized;
        transform.Translate(direction * speed * Time.deltaTime);
    }

    void Jump()
    {
        if (isGround == true)
        {
            isGround = false;
            rigid.AddForce(Vector3.up * jumpPower, ForceMode.Impulse);
        }
    }

    void OnCollisionEnter(Collision collision)
    {
        if (collision.gameObject.CompareTag("Ground"))
        {
            isGround = true;
        }
    }
}
```

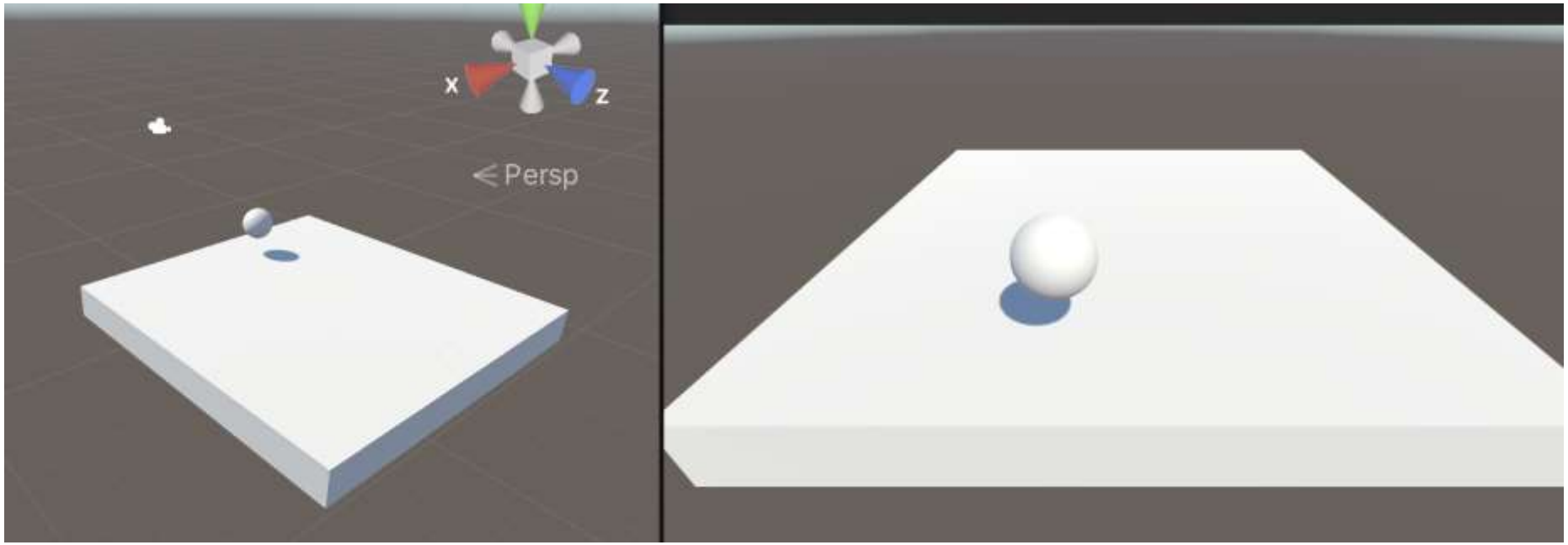


방향 벡터를 만들고
그곳으로 이동하는 방식
Vector3 (x,y,z)
(x,0,z)로 이동.

***normalized(정규화)**
벡터의 길이가 1인
단위벡터로 만들기
Why?
X=1, Z=1이면,
길이가 √2가 되잖슴~

***OnCollisionEnter : Rigidbody**컴포넌트가 **충돌 감지**하면 **알려주는 함수**

실행 잘 된다!



이제 총을 쏴보자

1. 플레이어랑 달라붙어서 같이 움직일 **총구** 만들기
2. 총구에서 나갈 **총알** 만들기
3. 총구랑 총알을 관리하고, 총알을 발사 할 **총 컨트롤러** 만들기

총구

1. 총알이 나갈 곳이니 플레이어랑 부딪히면 안 되고, 조금 떨어진 곳이어야 함

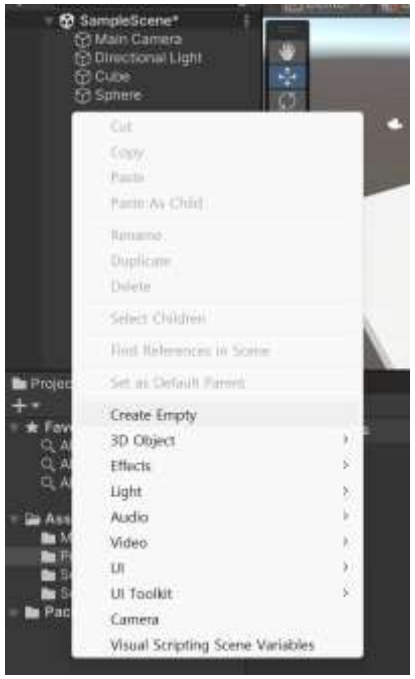
총알

1. 생성되는 즉시, 생성 방향으로 쭉 날라갔으면 좋겠음
2. 일정 거리 이상 가면, 사라졌으면 좋겠음.

총 컨트롤러

1. 총구랑 총알을 가지고 있어야 함
2. 매 프레임마다 확인해서, 좌클릭을 누르면, 총을 쐈으면 좋겠어.
3. 총을 쏜다는 의미는 총구에서 총알이 내가 지목한 방향으로 생성되는 거야

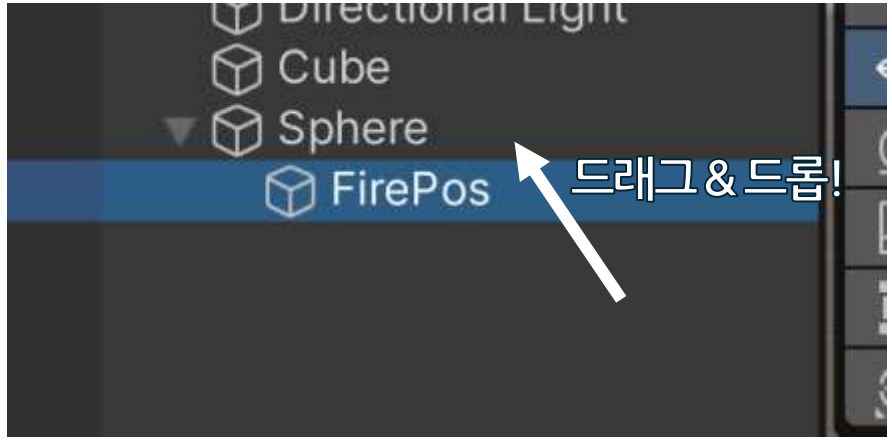
총구 생성



Create Empty 생성

모양 없이 그냥 Transform만 존재하는 오브젝트

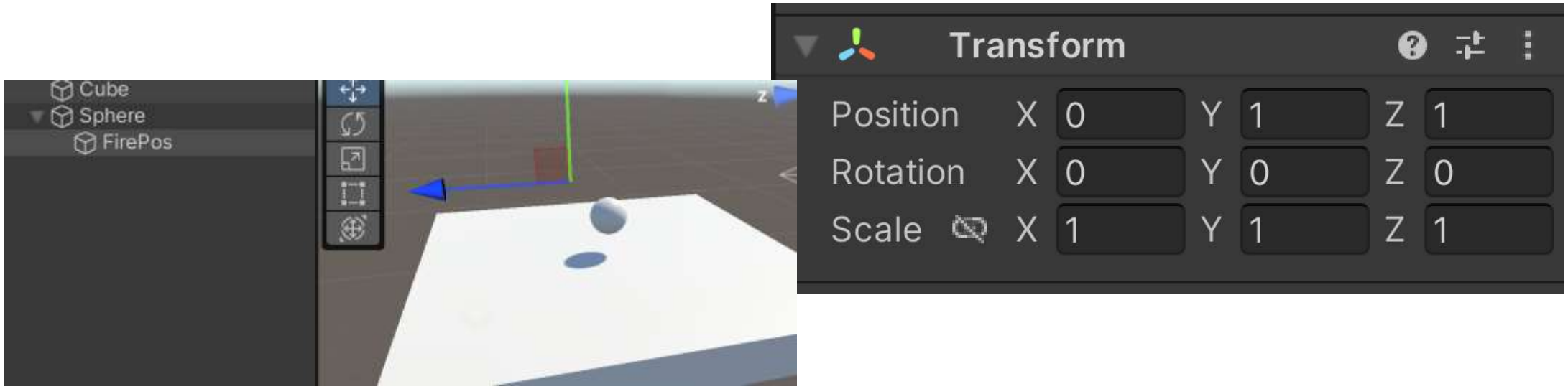
이름 **FirePos**로 설정 (총 쏠 위치)



Sphere에 **드래그&드롭**해 **자식 오브젝트**로 만들어버림

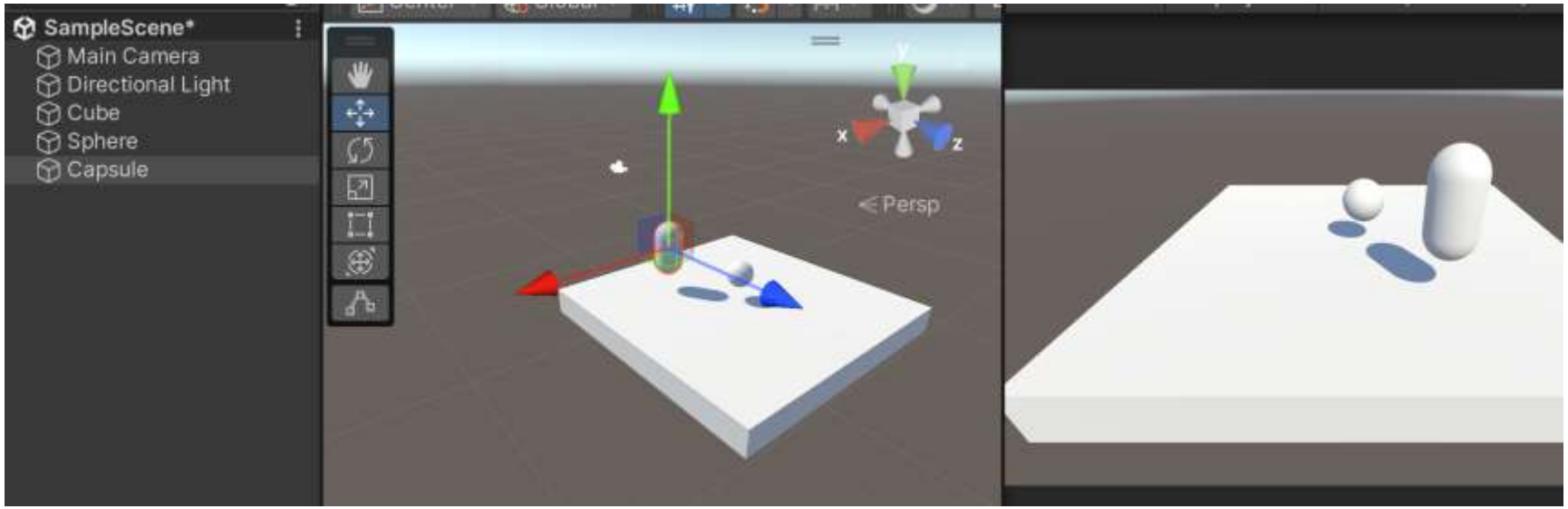
총구를 플레이어의 자식으로 만들면, 항상 같이 움직이게 할 수 있다.

총구 위치 설정



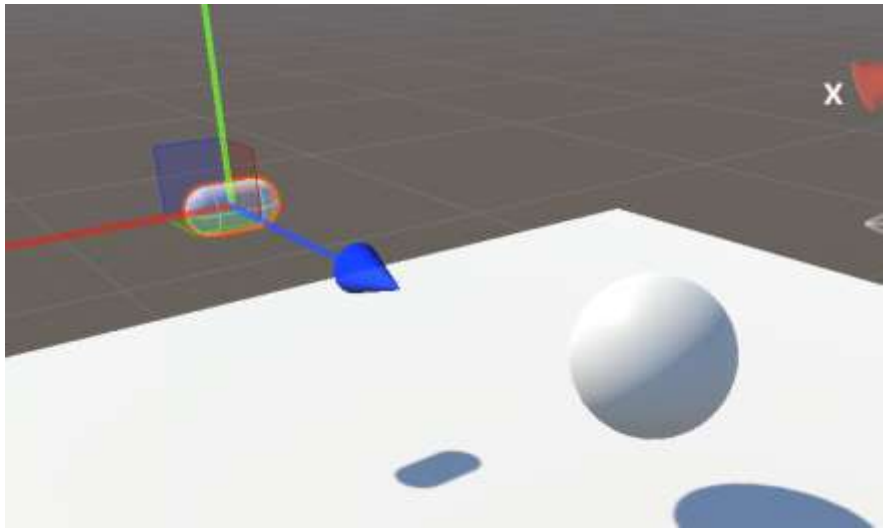
자식 오브젝트가 되었으니, 여기서 변경하는 Transform값은 부모 기준임.
부모 기준 Y축으로 +1, Z축으로 +1 하겠다는 의미

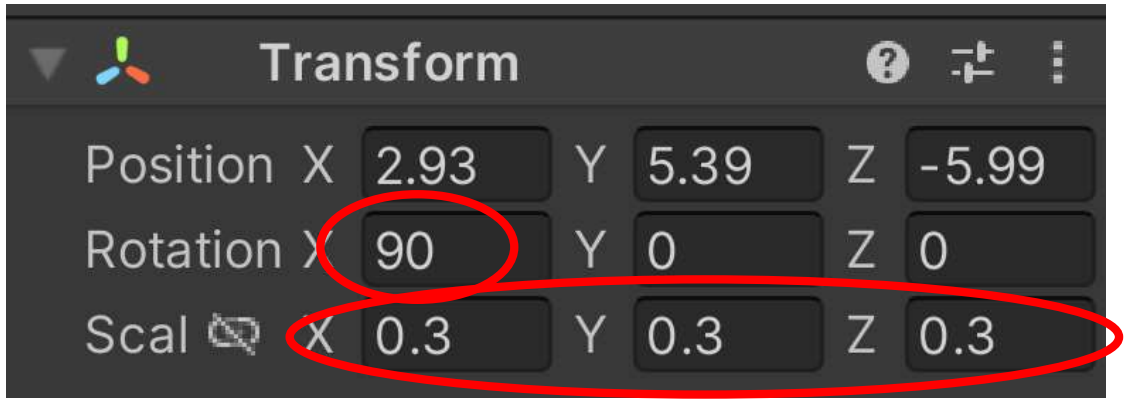
총알 모양 생성하기



계층 창 우클릭 - Create - 3D Object - **Capsule** 생성

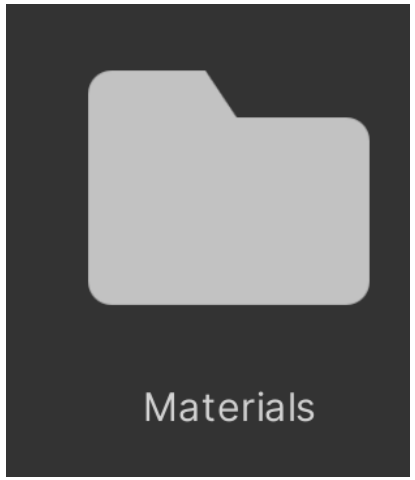
총알처럼 모양 변경



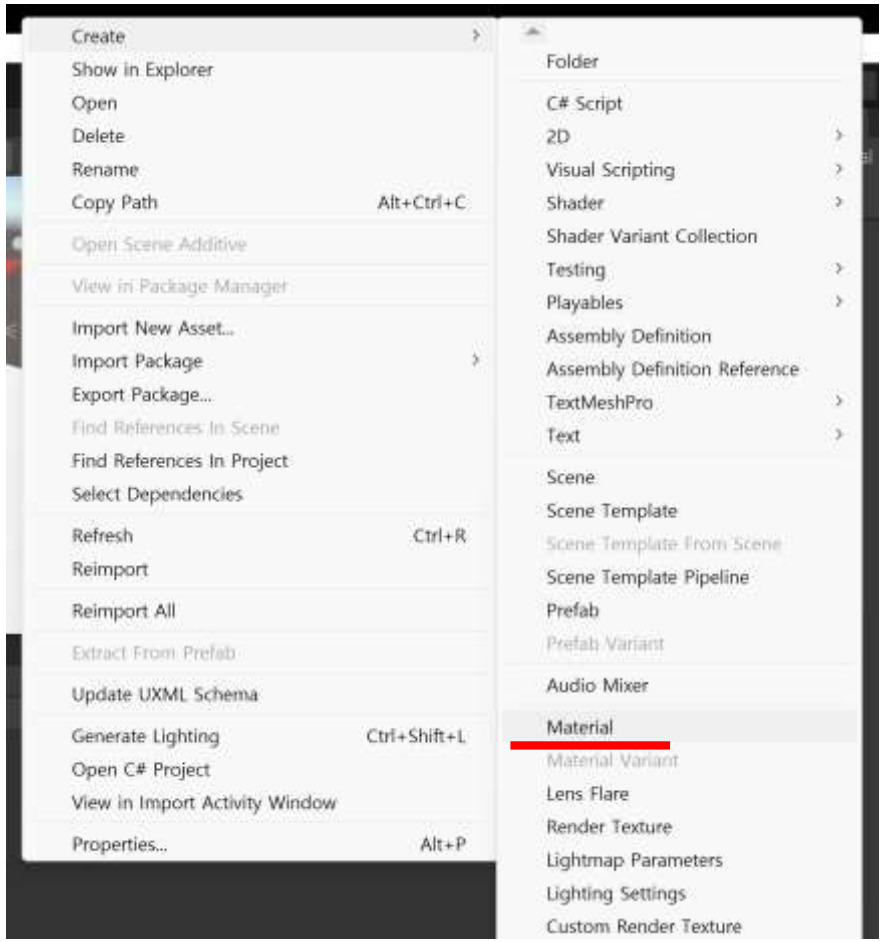


총알 모양처럼 작게 만들어보기

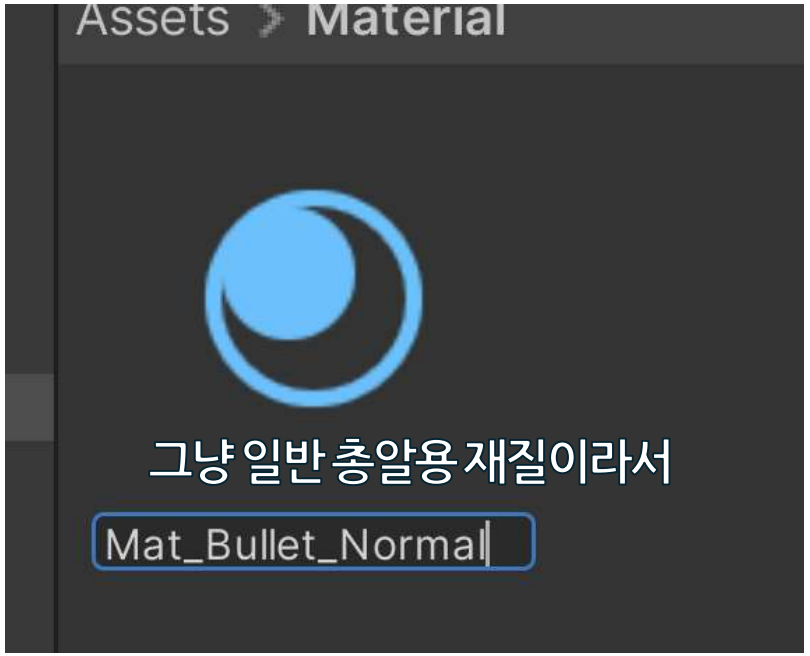
총알에 색칠 입힐 머테리얼 만들기



Materials 폴더 만들기



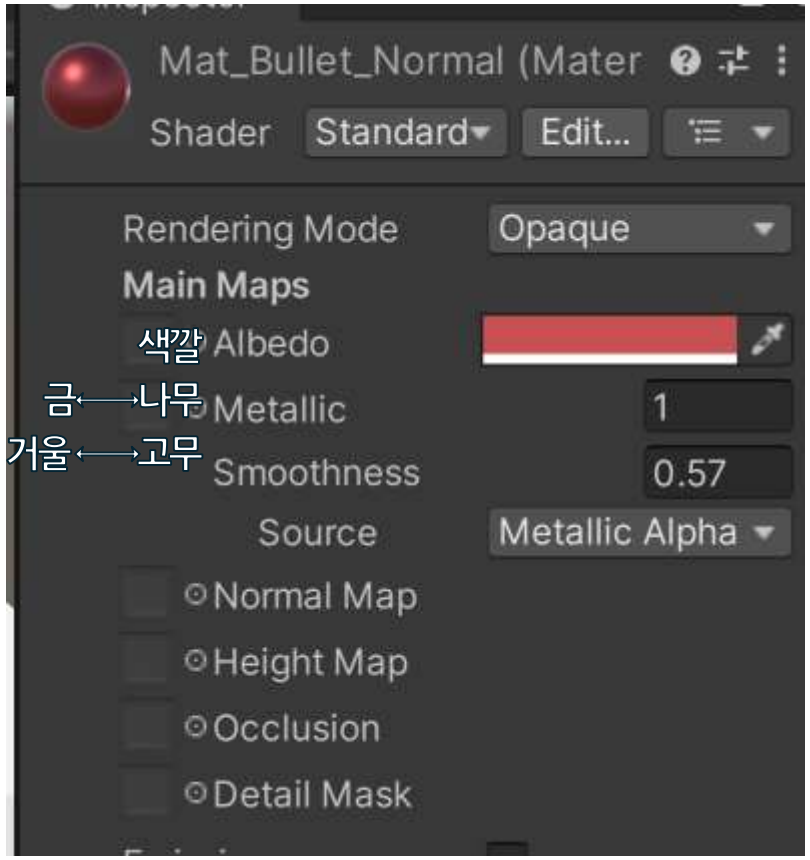
Create - Material 생성



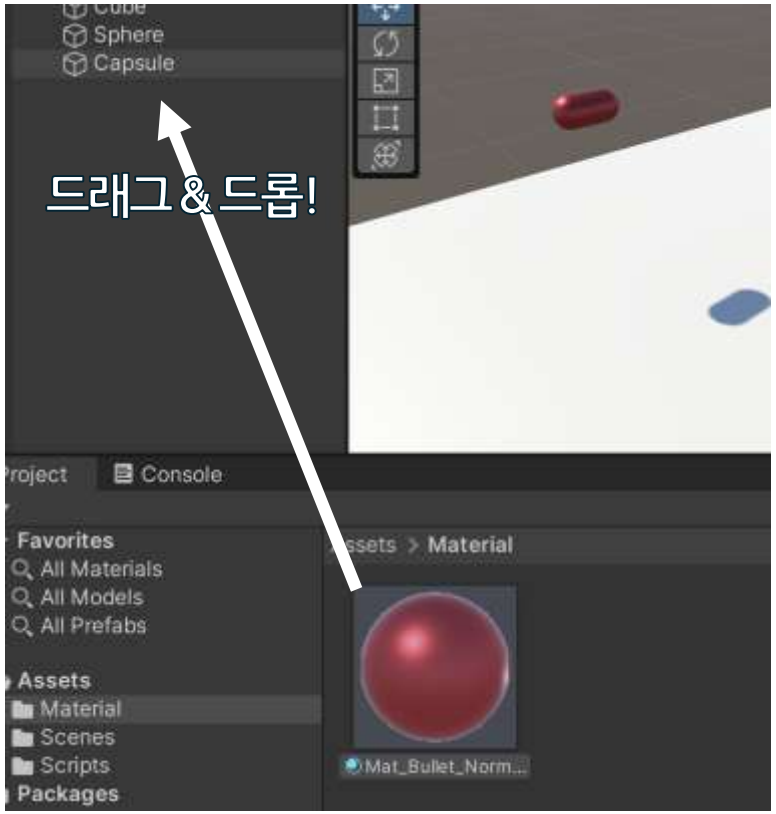
이름 **Mat_Bullet_Normal** 생성

Material은 대충 **색 입히는 컴포넌트**라고 생각

총알에 머테리얼 적용하기



적절히 설정 후 **Capsule에 드래그 & 드롭**

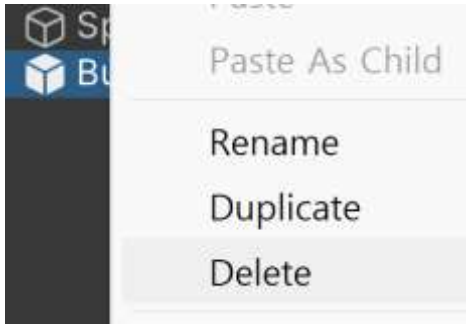


여기서는 가볍게 적용 정도만 다룹니다

총알을 이제 따로 데이터로 저장하자

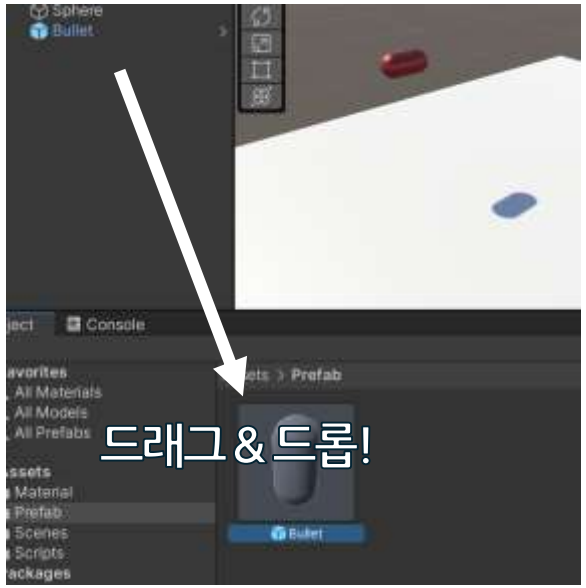
Prefabs 폴더 생성

Capsule 오브젝트 이름을 **Bullet**으로 설정



생성했으면 기존 Bullet은 헷갈리니까 그냥 지워버리자.

프로젝트 창으로 **드래그 & 드롭**해 바로 **프리팹 생성**



프리팹(Prefab)이란

"**Prefab**ricated Object" **미리 조립한 오브젝트**
하나 완벽하게 만들어놓고, 그걸 가져다 씀,
ex.총알을 만들었다→궤도, 속도, 충돌, 파괴 기능까지 세팅하고 필요할 때마다 꺼내 씀

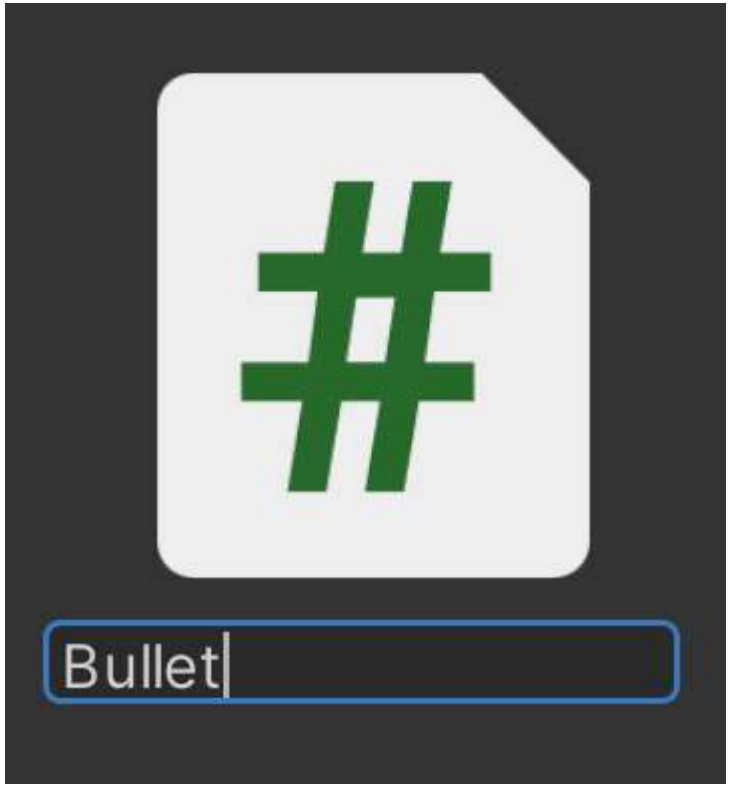
이러면 **오브젝트**를 **스크립트**에서 **생성**하는 게 가능해짐! ← 주 목적

총알 스크립트 만들기

Scripts폴더 - Create -C# Scripts -**Bullet.cs** 생성

총알

1. 생성되는 즉시, 생성 방향으로 쭉 날라갔으면 좋겠음
2. 일정 거리 이상 가면, 사라졌으면 좋겠음.



총 컨트롤러 스크립트 만들기

Scripts폴더 - Create -C# Scripts - **GunController.cs** 생성

총 컨트롤러

1. 총구랑 총알을 가지고 있어야 함
2. 매 프레임마다 확인해서, 좌클릭을 누르면, 총을 쐈으면 좋겠음
3. 총을 쏜다는 의미는 총구에서 총알이 내가 지목한 방향으로 생성되는 거야

총알 스크립트

시작할 때,
시작 위치를 기억해둠

방향 설정
외부에서 방향을 설정할 수 있게 함

매 프레임마다
설정된 **방향**으로 **이동**함
일정 거리 이상이면, 자체 삭제함

```
public class Bullet : MonoBehaviour
{
    private float bulletSpeed = 10f;
    private Vector3 startPos;
    private Vector3 direction;

    void Start()
    {
        startPos = transform.position;
    }

    public void SetDirection(Vector3 direction)
    {
        this.direction = direction;
    }

    void Update()
    {
        transform.position += direction * bulletSpeed * Time.deltaTime;

        float distance = Vector3.Distance(transform.position, startPos);
        if (distance > 5f)
        {
            Destroy(gameObject);
        }
    }
}
```

1-2 총알 스크립트

총 컨트롤러 스크립트

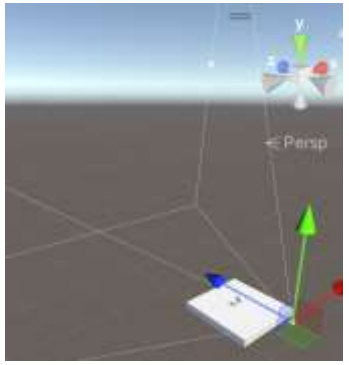
매 프레임마다
좌클릭 입력되면 쏜다

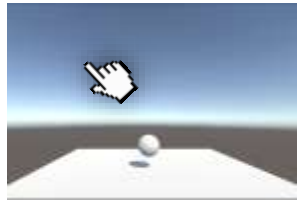
쏜다면.
마우스 방향을 가지는 벡터 얻기
총구 방향 설정
총알 발사 방향 계산
총알 생성

총알 나아갈 방향 설정

```
public class GunController : MonoBehaviour
{
    public GameObject bulletPrefab;
    public Transform firePos;

    void Update()
    {
        if (Input.GetMouseButton(0))
        {
            Shoot();
        }
    }

    void Shoot()
    {
        // 마우스 방향을 가지는 선 얻기
        Ray ray = Camera.main.ScreenPointToRay(Input.mousePosition);
        // 쏠 방향 설정
        firePos.rotation = Quaternion.LookRotation(ray.direction);
        // 총알 생성 방향 생성 (일반 각도가 아닌 Quaternion 방식을 사용하기 때문에 곱하기 연산)
        Quaternion fireDirection = firePos.rotation * bulletPrefab.transform.rotation;
        // 프리팹 복제품 생성(인스턴스 생성), 프리팹, 생성 위치, 생성 각도 인자가 필요함
        GameObject bulletObject = Instantiate(bulletPrefab, firePos.position, fireDirection);

        // 인스턴스에 붙어있는 Bullet 스크립트 가져오기
        Bullet bullet = bulletObject.GetComponent<Bullet>();
        // 해당 인스턴스의 방향을 직접 설정
        bullet.SetDirection(ray.direction);
    }
}
```

Ray 빔~ 개념





`ScreenPointToRay(`

화면의 점을 Ray로 바꿔줌

`Physics.Raycast(`

물체

(화면의 너비,높이)

`Ray ray`

(2,9)

`Camera.main`

메인 카메라(첫번째)

`Input.mousePosition`

화면의 마우스위치

1

`ray.direction`

방향을가지는크기1의벡터

`ray.origin`

벡터의시작점

(0,0)

메인 카메라의 화면

원래 메인 목적: 물체 감지

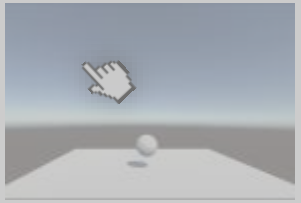
마우스가가리키는방향에 물체가있는지 알려줘!

Z축

*위치벡터는크기와 방향, 시작점이필요

```
Ray ray = Camera.main.ScreenPointToRay(Input.mousePosition);
```

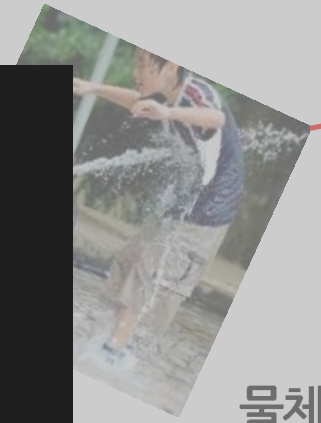
Ray 빔~ 개념



ScreenPointToRay(

화면의 점을 Ray로 비

Physics.Raycast(



물체

ray.direction

Camera.main

메인 카메라(첫번째)

ray.direc

방향을가지

ray.origin

벡터의 시작점

화면의 마우스 위치

우리가 쏠 녀석은 바로 이 놈

총알을 쏠 방향만 필요하다

마우스 방향대로 총알을 쏠 거니까!

원래 메인 목적: 물체 감지

마우스가 가리키는 방향에 물체가 있는지 알려줘!

(0,0)

메인 카메라의 화면

Z축

*위치벡터는 **크기**와 **방향**, **시작점**이 필요

```
Ray ray = Camera.main.ScreenPointToRay(Input.mousePosition);
```

Quaternion

컴퓨터에서 3D 공간을 회전시킬 때 편한 변환 도구

우리가 편히 아는 각 **eulerAngles(Vector3)**를 써도 되지만, 축이 꼬이거나 회전이 이상하게 되는 경우가 생김.

그렇기 때문에 **Vector3**의 **rotation**은 **Quarternion**을 사용함

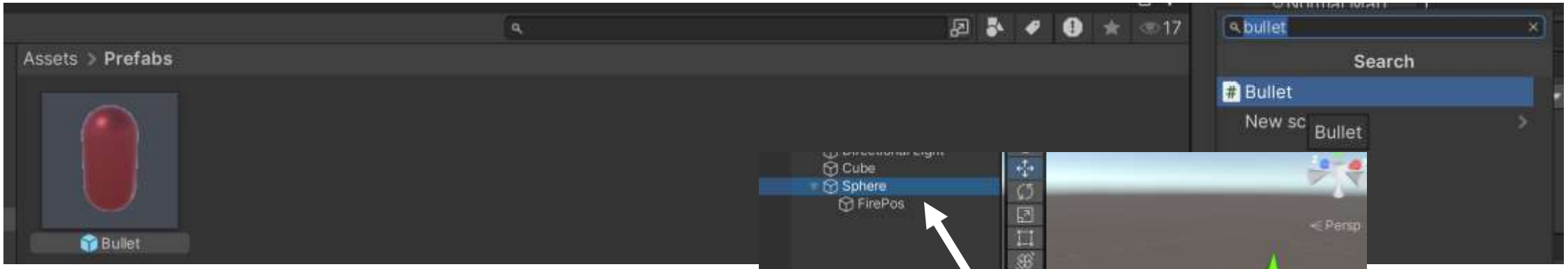
Instantiate에 들어갈 **인스턴스 생성 각도**를 **Vector3**에서 **Quarternion**으로 변환해서 사용할 필요가 있음

```
firePos.rotation =
```

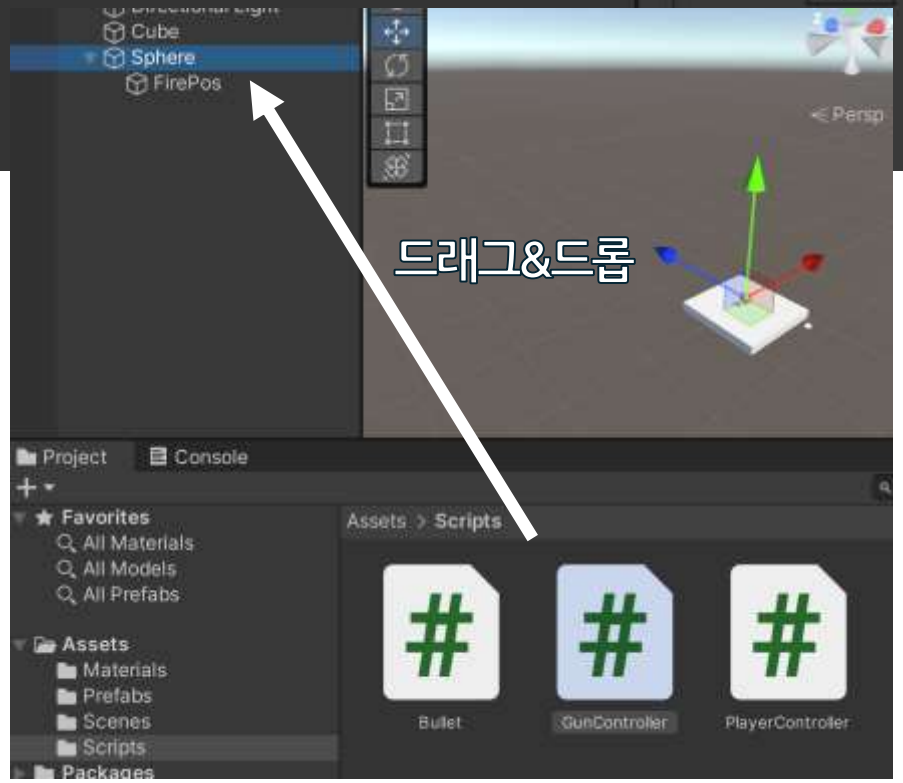
겸사겸사 총구 방향도 돌리기

```
Quaternion.LookRotation(ray.direction)
```

스크립트 적용



Bullet 프리팹에 AddComponent - Bullet 검색 후 선택

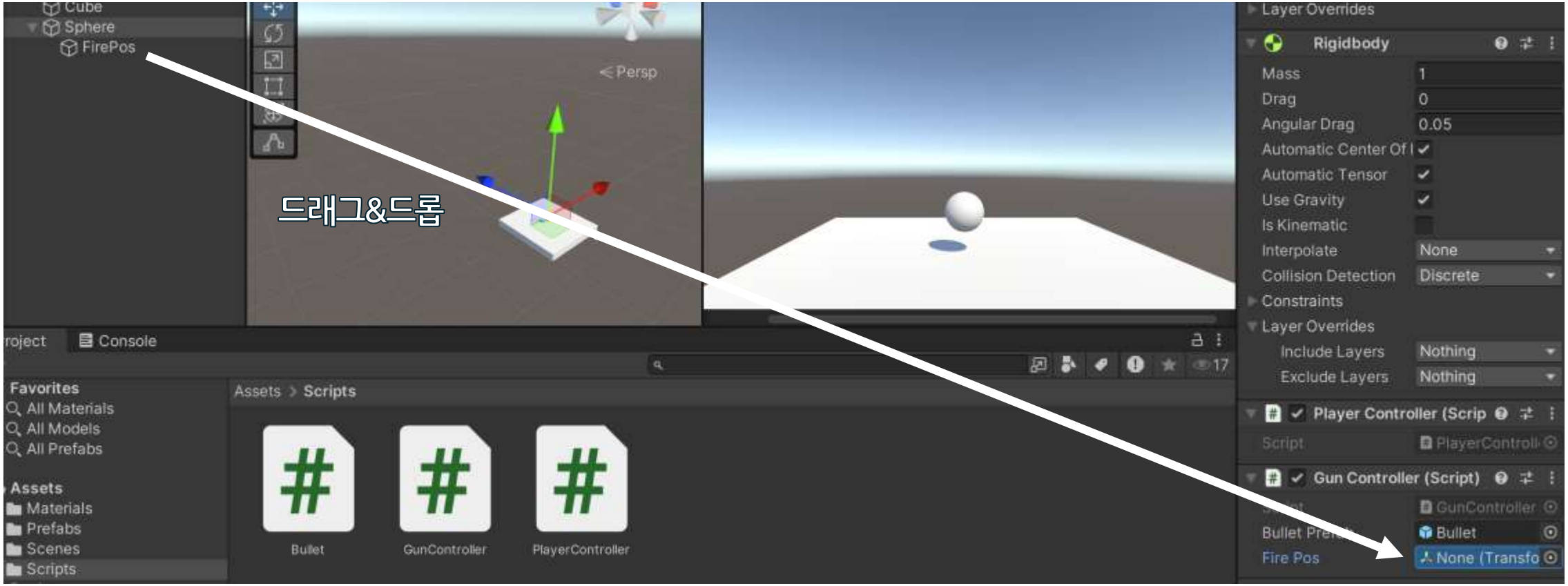


Sphere 오브젝트에 GunController.cs 드래그 &드롭

총 컨트롤러 변수 설정하기

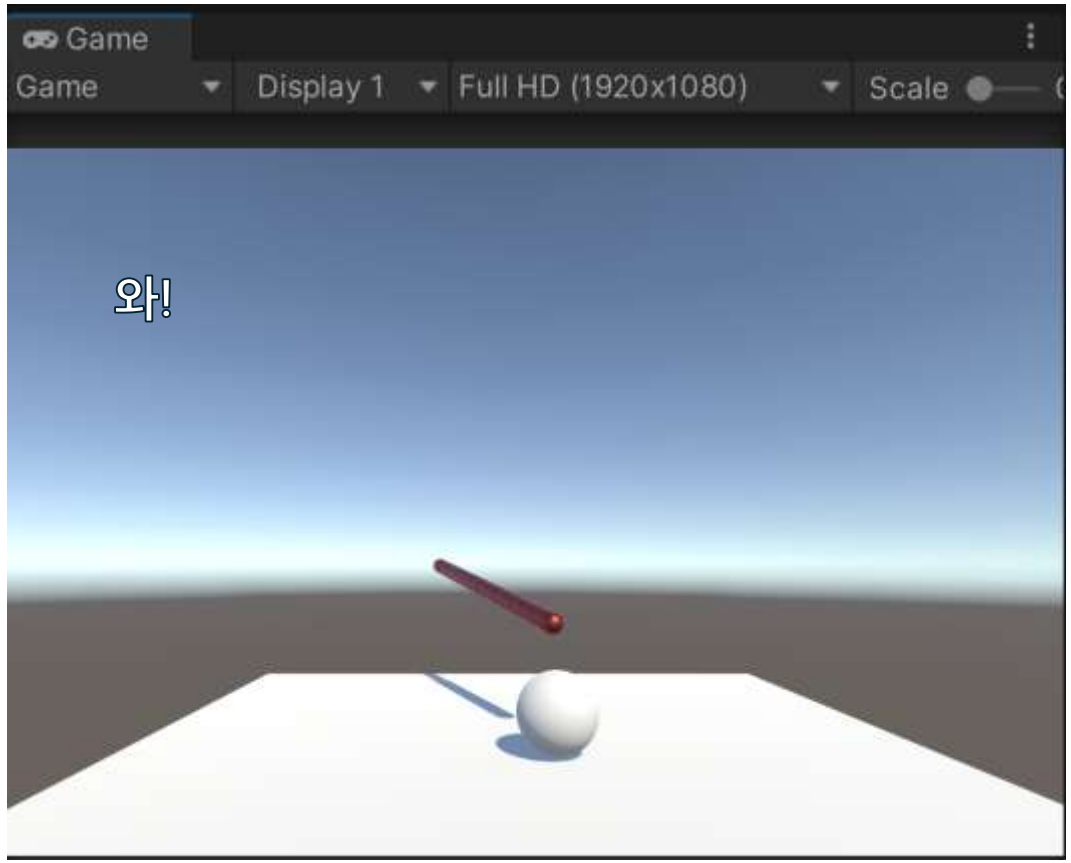
Bullet프리팹을 Bullet(GameObject)변수 칸에 **드래그&드롭**

총 컨트롤러 변수 설정하기



FirePos 오브젝트를 FirePos(Transform) 변수 칸에 **드래그&드롭**

잘 동작한다!



축하합니다

이제 당신은 **컴포넌트**, **스크립트**, **Ray**, **머테리얼**, **Transform**, **카메라**, **Rigidbody**, **프리팹**, **인스턴시에이션** 등을 학습했어요.